University of Asia Pacific

Department of Computer Science and Engineering

Course Code : CSE 450 (VLSI Lab)

Final Project | Spring 2026

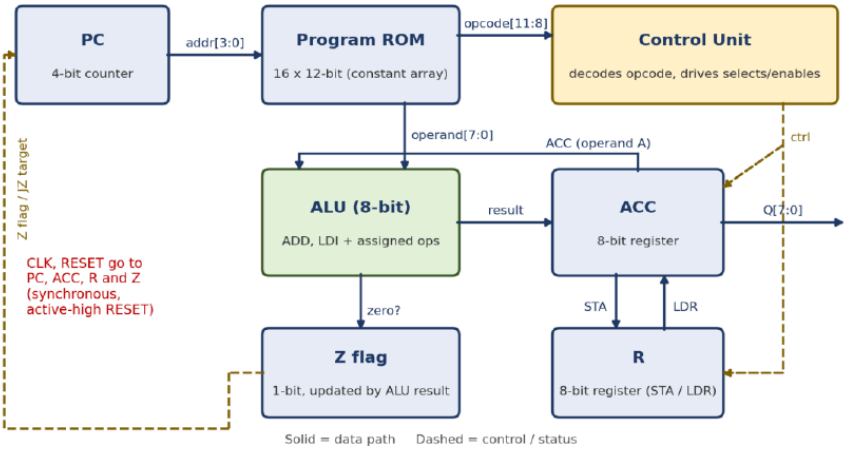
Total Marks	50 (Viva → 30, Report & VrC → 10, Presentation → 10)		
Student Name	Rifah Tasfia (Section C)	**ID**	22201126

Final Project: Mini 8-bit Accumulator CPU

Objective. Design and implement a small single-cycle 8-bit accumulator CPU in VHDL. The CPU supports only five instructions: two are mandatory for every student, and the remaining three are assigned to you according to your Student ID (see the “Instruction Set Assignment” page: the last digit of your Student ID decides your set). You must also verify the design with a testbench that runs the program given for your set in Appendix B.

1. System Architecture

The CPU is built from the blocks shown below. Every sequential block (PC, ACC, R, Z) is clocked on the rising edge of CLK and cleared by a synchronous, active-high RESET.



Block	Description
PC	4-bit program counter. Holds the address of the instruction being executed. After every instruction it becomes PC + 1, except when JZ is taken.
Program ROM	16 locations x 12 bits, written as a constant array in VHDL. It is read combinationally using PC (no clock needed). Unused locations contain 0x000 (NOP).
Control Unit	Decodes opcode[11:8] and generates the ALU function select and the load/enable signals for ACC, R, Z and PC.
ALU	8-bit combinational unit working on ACC and the 8-bit operand. Only the operations of your own instruction set are required.
ACC, R, Z	ACC: 8-bit accumulator, also the CPU output Q[7:0]. R: 8-bit holding register (only if your set has STA / LDR). Z: 1-bit flag, 1 when the value written to ACC is 0x00.

2. Instruction Format

Each instruction is 12 bits: **instr[11:8] = opcode** (4 bits) and **instr[7:0] = operand** (8-bit immediate). Every instruction takes exactly one clock cycle (fetch, decode and execute happen in the same cycle, the result is written at the rising edge).

Mandatory instructions (all students):

Mnemonic	Opcode	Operation	Flags
LDI imm	0001	ACC ← imm	Z updated
ADD imm	0010	ACC ← ACC + imm (8-bit, carry discarded)	Z updated

Instruction pool (you implement only the three assigned to you):

Mnemonic	Opcode	Operation	Flags
SUB imm	0011	ACC ← ACC − imm (modulo 256)	Z updated
AND imm	0100	ACC ← ACC AND imm	Z updated
OR imm	0101	ACC ← ACC OR imm	Z updated
XOR imm	0110	ACC ← ACC XOR imm	Z updated
NOT	0111	ACC ← NOT ACC (operand ignored)	Z updated
SHL	1000	ACC ← ACC shifted left by 1, 0 enters LSB (operand ignored)	Z updated
SHR	1001	ACC ← ACC shifted right by 1, 0 enters MSB (operand ignored)	Z updated
STA	1010	R ← ACC (operand ignored, ACC unchanged)	Z unchanged
LDR	1011	ACC ← R (operand ignored)	Z updated
JZ addr	1100	If Z = 1 then PC ← operand[3:0], else PC ← PC + 1	Z unchanged

Note: if your set contains LDR it also contains STA. Any opcode not in your set (including 0000) behaves as NOP: PC ← PC + 1 and nothing else changes.

3. Reset Behaviour

RESET = 1 at a rising edge: PC ← 0, ACC ← 0x00, R ← 0x00, Z ← 1 (RESET has priority over everything). **RESET = 0 at a rising edge:** execute the instruction at ROM[PC]. Between clock edges all registers keep their values.

4. Implementation Requirements

- Use VHDL with IEEE.STD_LOGIC_1164.ALL (IEEE.NUMERIC_STD is allowed for arithmetic).
- Modular design: separate entities for PC, Program ROM, Control Unit, ALU and ACC/R/Z, connected in a top-level CPU entity.
- RESET must be synchronous and active-high. All state changes must occur on the rising edge of CLK.
- The top level must expose CLK, RESET, Q[7:0] (the ACC), the Z flag and, for checking, PC[3:0].
- Implement only the five instructions of your own set. The ROM must hold exactly the Appendix B program of your set; the testbench must generate a clock, apply RESET for one rising edge, and run until the last instruction has executed.

Instruction Set Assignment

How to find your set: take the **last digit of your Student ID** (ID mod 10). That digit is your set number in the table below. Example: ID 22201014 → last digit 4 → Set 4.

Every student implements the two mandatory instructions **LDI** and **ADD** plus the three assigned instructions of his/her set. Total = 5 instructions.

Last digit of ID	Set	Mandatory	Assigned	Complete instruction set of the CPU
0	**Set 0**	LDI, ADD	AND, JZ, NOT	LDI, ADD, AND, JZ, NOT
1	**Set 1**	LDI, ADD	SUB, AND, OR	LDI, ADD, SUB, AND, OR
2	**Set 2**	LDI, ADD	XOR, NOT, SHL	LDI, ADD, XOR, NOT, SHL
3	**Set 3**	LDI, ADD	SHR, SUB, XOR	LDI, ADD, SHR, SUB, XOR
4	**Set 4**	LDI, ADD	STA, LDR, AND	LDI, ADD, STA, LDR, AND

Last digit of ID	Set	Mandatory	Assigned	Complete instruction set of the CPU
5	**Set 5**	LDI, ADD	OR, NOT, SHR	LDI, ADD, OR, NOT, SHR
6	**Set 6**	LDI, ADD	SUB, JZ, SHL	LDI, ADD, SUB, JZ, SHL
7	**Set 7**	LDI, ADD	AND, XOR, SHR	LDI, ADD, AND, XOR, SHR
8	**Set 8**	LDI, ADD	STA, LDR, SUB	LDI, ADD, STA, LDR, SUB
9	**Set 9**	LDI, ADD	NOT, SHL, JZ	LDI, ADD, NOT, SHL, JZ

Appendix B: Required Test Program and Expected Results

Store the program of your set (Appendix B tables are ordered by set number 0–9) in the Program ROM. Apply RESET for one rising edge, then run the CPU for one clock cycle per executed instruction. The ACC, Z, R and next-PC columns show the values after the corresponding rising edge. Addresses that are not listed contain 0x000 (NOP). Operands are in hexadecimal. For NOT, SHL, SHR, STA and LDR the operand field is 0x00.

ROM encoding: instr[11:0] = opcode (4 bit) & operand (8 bit). Example: LDI 0x3C = 0001 00111100 = 0x13C.

Set 0: LDI, ADD, AND, JZ, NOT

ROM contents: 0: LDI 0xA5 → 0x1A5 | 1: AND 0x5A → 0x45A | 2: JZ 0x05 → 0xC05 | 3: ADD 0x01 → 0x201 | 4: ADD 0x01 → 0x201 | 5: NOT → 0x700 | 6: AND 0x0F → 0x40F | 7: JZ 0x00 → 0xC00 | 8: ADD 0x01 → 0x201

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0xA5	0xA5	0	10100101	1
2	1	AND 0x5A	0x00	1	00000000	2
3	2	JZ 0x05 (taken)	0x00	1	00000000	5
4	5	NOT	0xFF	0	11111111	6
5	6	AND 0x0F	0x0F	0	00001111	7
6	7	JZ 0x00 (not taken)	0x0F	0	00001111	8
7	8	ADD 0x01	0x10	0	00010000	9

Set 1: LDI, ADD, SUB, AND, OR

ROM contents: 0: LDI 0x3C → 0x13C | 1: ADD 0x0F → 0x20F | 2: SUB 0x0B → 0x30B | 3: AND 0xF0 → 0x4F0 | 4: OR 0x0A → 0x50A | 5: SUB 0x4A → 0x34A | 6: SUB 0x01 → 0x301

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x3C	0x3C	0	00111100	1
2	1	ADD 0x0F	0x4B	0	01001011	2
3	2	SUB 0x0B	0x40	0	01000000	3
4	3	AND 0xF0	0x40	0	01000000	4
5	4	OR 0x0A	0x4A	0	01001010	5
6	5	SUB 0x4A	0x00	1	00000000	6
7	6	SUB 0x01	0xFF	0	11111111	7

Set 2: LDI, ADD, XOR, NOT, SHL

ROM contents: 0: LDI 0x55 → 0x155 | 1: XOR 0xFF → 0x6FF | 2: NOT → 0x700 | 3: SHL → 0x800 | 4: ADD 0x56 → 0x256 | 5: XOR 0x81 → 0x681 | 6: SHL → 0x800

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x55	0x55	0	01010101	1
2	1	XOR 0xFF	0xAA	0	10101010	2
3	2	NOT	0x55	0	01010101	3
4	3	SHL	0xAA	0	10101010	4
5	4	ADD 0x56	0x00	1	00000000	5
6	5	XOR 0x81	0x81	0	10000001	6
7	6	SHL	0x02	0	00000010	7

Set 3: LDI, ADD, SHR, SUB, XOR

ROM contents: 0: LDI 0x80 → 0x180 | 1: SHR → 0x900 | 2: SUB 0x10 → 0x310 | 3: ADD 0x0F → 0x20F | 4: XOR 0x3F → 0x63F | 5: ADD 0x07 → 0x207 | 6: SHR → 0x900

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x80	0x80	0	10000000	1
2	1	SHR	0x40	0	01000000	2
3	2	SUB 0x10	0x30	0	00110000	3
4	3	ADD 0x0F	0x3F	0	00111111	4
5	4	XOR 0x3F	0x00	1	00000000	5
6	5	ADD 0x07	0x07	0	00000111	6
7	6	SHR	0x03	0	00000011	7

Set 4: LDI, ADD, STA, LDR, AND

ROM contents: 0: LDI 0x1E → 0x11E | 1: STA → 0xA00 | 2: ADD 0x22 → 0x222 | 3: AND 0x60 → 0x460 | 4: LDR → 0xB00 | 5: ADD 0x01 → 0x201 | 6: AND 0x0F → 0x40F | 7: STA → 0xA00 | 8: LDI 0x00 → 0x100 | 9: LDR → 0xB00

Cycle	PC	Instruction executed	ACC (hex)	Z	R (hex)	ACC (bin)	Next PC
0	-	RESET = 1	0x00	1	0x00	00000000	0
1	0	LDI 0x1E	0x1E	0	0x00	00011110	1
2	1	STA	0x1E	0	0x1E	00011110	2
3	2	ADD 0x22	0x40	0	0x1E	01000000	3
4	3	AND 0x60	0x40	0	0x1E	01000000	4
5	4	LDR	0x1E	0	0x1E	00011110	5
6	5	ADD 0x01	0x1F	0	0x1E	00011111	6
7	6	AND 0x0F	0x0F	0	0x1E	00001111	7
8	7	STA	0x0F	0	0x0F	00001111	8
9	8	LDI 0x00	0x00	1	0x0F	00000000	9
10	9	LDR	0x0F	0	0x0F	00001111	10

Set 5: LDI, ADD, OR, NOT, SHR

ROM contents: 0: LDI 0x0F → 0x10F | 1: OR 0xA0 → 0x5A0 | 2: NOT → 0x700 | 3: SHR → 0x900 | 4: ADD 0xD8 → 0x2D8 | 5: OR 0x81 → 0x581 | 6: SHR → 0x900

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x0F	0x0F	0	00001111	1
2	1	OR 0xA0	0xAF	0	10101111	2
3	2	NOT	0x50	0	01010000	3
4	3	SHR	0x28	0	00101000	4
5	4	ADD 0xD8	0x00	1	00000000	5
6	5	OR 0x81	0x81	0	10000001	6
7	6	SHR	0x40	0	01000000	7

Set 6: LDI, ADD, SUB, JZ, SHL

ROM contents: 0: LDI 0x02 → 0x102 | 1: SUB 0x01 → 0x301 | 2: JZ 0x07 → 0xC07 | 3: SUB 0x01 → 0x301 | 4: JZ 0x07 → 0xC07 | 5: ADD 0xAA → 0x2AA | 6: ADD 0xAA → 0x2AA | 7: LDI 0x21 → 0x121 | 8: SHL → 0x800 | 9: ADD 0x01 → 0x201

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x02	0x02	0	00000010	1
2	1	SUB 0x01	0x01	0	00000001	2
3	2	JZ 0x07 (not taken)	0x01	0	00000001	3
4	3	SUB 0x01	0x00	1	00000000	4
5	4	JZ 0x07 (taken)	0x00	1	00000000	7
6	7	LDI 0x21	0x21	0	00100001	8
7	8	SHL	0x42	0	01000010	9
8	9	ADD 0x01	0x43	0	01000011	10

Set 7: LDI, ADD, AND, XOR, SHR

ROM contents: 0: LDI 0xF3 → 0x1F3 | 1: AND 0x3C → 0x43C | 2: XOR 0x11 → 0x611 | 3: SHR → 0x900 | 4: ADD 0x30 → 0x230 | 5: XOR 0x40 → 0x640

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0xF3	0xF3	0	11110011	1
2	1	AND 0x3C	0x30	0	00110000	2
3	2	XOR 0x11	0x21	0	00100001	3
4	3	SHR	0x10	0	00010000	4
5	4	ADD 0x30	0x40	0	01000000	5
6	5	XOR 0x40	0x00	1	00000000	6

Set 8: LDI, ADD, STA, LDR, SUB

ROM contents: 0: LDI 0x64 → 0x164 | 1: STA → 0xA00 | 2: SUB 0x14 → 0x314 | 3: ADD 0x0A → 0x20A | 4: STA → 0xA00 | 5: SUB 0x5A → 0x35A | 6: LDR → 0xB00 | 7: SUB 0x60 → 0x360

Cycle	PC	Instruction executed	ACC (hex)	Z	R (hex)	ACC (bin)	Next PC
0	-	RESET = 1	0x00	1	0x00	00000000	0
1	0	LDI 0x64	0x64	0	0x00	01100100	1
2	1	STA	0x64	0	0x64	01100100	2
3	2	SUB 0x14	0x50	0	0x64	01010000	3
4	3	ADD 0x0A	0x5A	0	0x64	01011010	4
5	4	STA	0x5A	0	0x5A	01011010	5
6	5	SUB 0x5A	0x00	1	0x5A	00000000	6
7	6	LDR	0x5A	0	0x5A	01011010	7
8	7	SUB 0x60	0xFA	0	0x5A	11111010	8

Set 9: LDI, ADD, NOT, SHL, JZ

ROM contents: 0: LDI 0xFF → 0x1FF | 1: NOT → 0x700 | 2: JZ 0x05 → 0xC05 | 3: ADD 0x77 → 0x277 | 4: ADD 0x77 → 0x277 | 5: ADD 0x11 → 0x211 | 6: SHL → 0x800 | 7: JZ 0x00 → 0xC00 | 8: SHL → 0x800

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0xFF	0xFF	0	11111111	1
2	1	NOT	0x00	1	00000000	2
3	2	JZ 0x05 (taken)	0x00	1	00000000	5
4	5	ADD 0x11	0x11	0	00010001	6
5	6	SHL	0x22	0	00100010	7
6	7	JZ 0x00 (not taken)	0x22	0	00100010	8
7	8	SHL	0x44	0	01000100	9

Submission Guidelines

Submit your VHDL source files, testbench and a project report within the 2-week period. Include this project description (question, assignment page and Appendix B) at the front of your report and start your own report on a new page after it. The report must include, but is not limited to, the following points.

- Your Student ID, assigned set number and the three assigned instructions (highlighted in yellow color)
- Attach your entity code (all modules), testbench code and output visuals (waveform with PC, ACC, Z and R if used)
- Challenges faced during the experiment, and conclusion

Project Report

Mini 8-bit Accumulator CPU in VHDL

CSE 450 (VLSI Lab) | Final Project | Spring 2026

1. Student Information and Assigned Instruction Set

Item	Details
Student Name	Rifah Tasfia
Student ID	22201126 (last digit = 6)
Section	C1
Assigned set	Set 6
Mandatory instructions	LDI, ADD
Assigned instructions	SUB, JZ, SHL
Complete instruction set	LDI, ADD, SUB, JZ, SHL
Language / tools	VHDL-93 (IEEE.STD_LOGIC_1164; IEEE.NUMERIC_STD only in testbenches), Xilinx ISE 14.7 project (XST / ISim). Waveforms in this report are from the testbench runs of the final code.

The CPU implements five instructions, all single-cycle with a 12-bit format (opcode = instr[11:8], operand = instr[7:0]). The three instructions assigned to Set 6 are highlighted in Table 1.

Table 1: *Instruction set implemented by this CPU (Set 6)*

Mnemonic	Opcode	Operation	Flags
LDI imm	0001	ACC <- imm	Z updated
ADD imm	0010	ACC <- ACC + imm (8-bit, carry discarded)	Z updated
SUB imm	0011	ACC <- ACC - imm (modulo 256)	Z updated
SHL	1000	ACC <- ACC shifted left by 1, 0 enters LSB (operand ignored)	Z updated
JZ addr	1100	If Z = 1 then PC <- operand[3:0], else PC <- PC + 1	Z unchanged

2. Design Overview

The processor was designed bottom-up. The only primitive gate is a 2-input NAND gate; NOT, AND, OR, NOR, XOR and XNOR gates are built from it, the gates are combined into half adders, full adders and an 8-bit ripple-carry adder, and these are used to build the ALU, the zero detector, the program counter and finally the complete CPU. Every module has its own testbench, and the top-level testbench runs the Set 6 program of Appendix B and checks PC, ACC and Z after every clock edge with assertions.

2.1 Module hierarchy

Table 2: *Module hierarchy (lowest level first)*

Level	Module	Built from	Role
0	NAND_gate	Dataflow: Y <= A nand B	Only primitive gate
1	not_gate, and_gate, or_gate, nor_gate, xor_gate, xnor_gate	NAND gates only	Basic logic
2	half_adder, full_adder	XOR / AND / OR gates	1-bit arithmetic
3	bit_8_full_adder	8 full adders (ripple carry)	8-bit adder

Level	Module	Built from	Role
3	D_flip_flop, register_8_bit	Clocked process (sync. reset); 8 D flip-flops	Storage
4	alu_8bit, zero_flag, accumulator_8bit, program_counter_4_bit	Level 1 to 3 modules	Datapath blocks
4	program_rom, instruction_decoder, control_unit	Behavioural case statements + dataflow logic	Program storage and control
5	cpu_top	All of the above	Complete CPU

2.2 CPU architecture

Figure 1 shows how the blocks of the final design are connected. The Program Counter addresses the Program ROM; the 12-bit instruction goes to the Control Unit, whose instruction decoder splits it into opcode and operand and raises one enable line (LOAD, ADD, SUB, JZ or SHL). From these enables the control unit generates the ALU function select (ALU_SEL), the accumulator source select (ACC_SRC), the write enable (ACC_WE) and the jump signal PC_LOAD = JZ_EN AND Z. The ALU works on ACC (input A) and the operand (input B). The ACC source multiplexer passes the operand for LDI and the ALU result for every other instruction, and the accumulator stores this value at the rising clock edge. The zero detector checks the same value, so the Z flip-flop is written together with ACC; JZ and NOP write neither ACC nor Z. The program counter loads the jump target operand[3:0] when PC_LOAD = 1 and becomes PC + 1 otherwise.

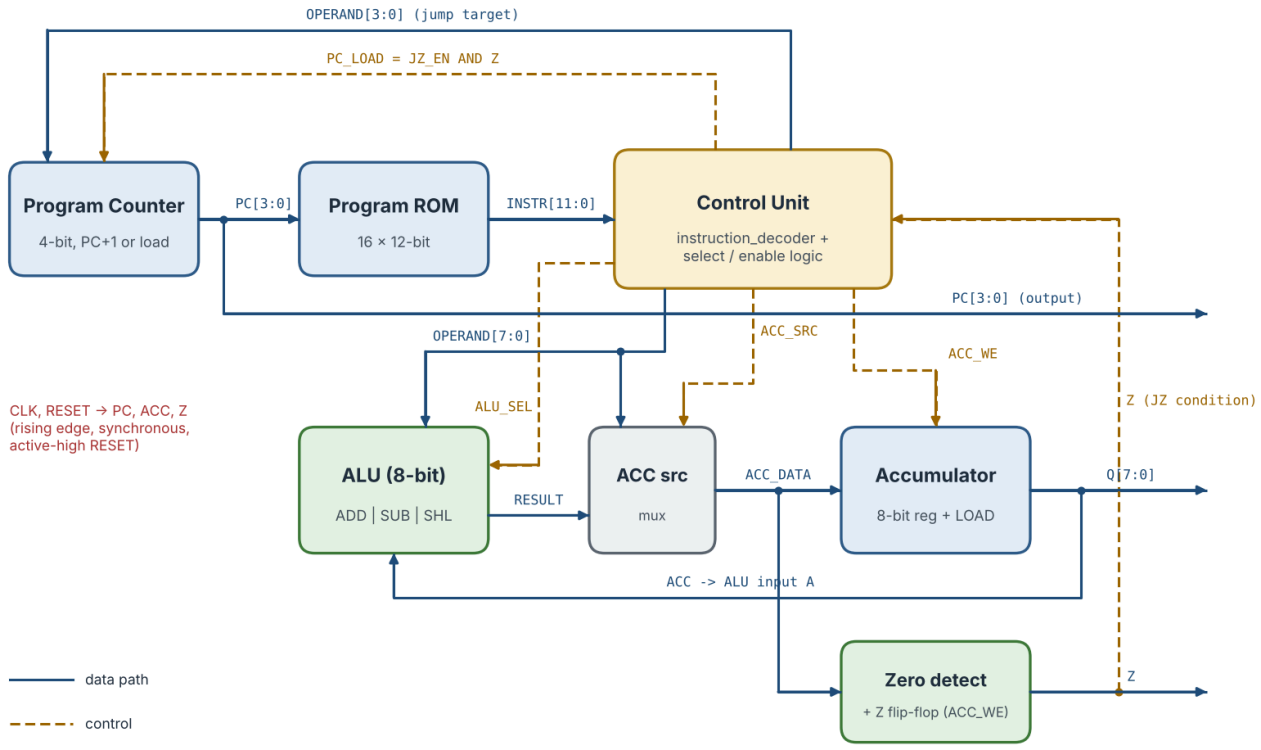


Figure 1: *Block diagram of the implemented 8-bit accumulator CPU (Set 6)*

2.3 Program stored in the ROM

The Program ROM holds exactly the Appendix B program of Set 6. Unused locations return 0x000, which is decoded as NOP (no register is written, PC <- PC + 1).

Table 3: *Program ROM contents (Set 6)*

Address	ROM word	Instruction	Encoding (opcode & operand)
0	0x102	LDI 0x02	0001 & 00000010
1	0x301	SUB 0x01	0011 & 00000001
2	0xC07	JZ 0x07	1100 & 00000111
3	0x301	SUB 0x01	0011 & 00000001
4	0xC07	JZ 0x07	1100 & 00000111
5	0x2AA	ADD 0xAA	0010 & 10101010
6	0x2AA	ADD 0xAA	0010 & 10101010
7	0x121	LDI 0x21	0001 & 00100001
8	0x800	SHL	1000 & 00000000
9	0x201	ADD 0x01	0010 & 00000001
10 - 15	0x000	NOP	0000 & 00000000

3. Basic Building Blocks

These small blocks are the foundation of the datapath. Each one has its own testbench; the gate and adder outputs were checked against their truth tables. Repeated component declarations are omitted from the listings below to save space.

3.1 NAND gate

The only primitive of the design. All other gates are made from it.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity NAND_gate is
    Port (
        A : in  STD_LOGIC;
        B : in  STD_LOGIC;
        Y : out STD_LOGIC
    );
end NAND_gate;

architecture Dataflow of NAND_gate is

begin

    Y <= A NAND B;

end Dataflow;
```

3.2 NOT gate

A NAND gate with both inputs tied together: Y = NOT A.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity not_gate is
    Port (
        A : in STD_LOGIC;
        Y : out STD_LOGIC
    );
end not_gate;

architecture Structural of not_gate is
    -- component declarations omitted: NAND_gate

begin

    NAND1: NAND_gate
        port map (
            A => A,
            B => A,
```

```
            Y => Y
        );

end Structural;
```

3.3 AND gate

A NAND gate followed by a NAND used as an inverter.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity and_gate is
    Port (
        A : in STD_LOGIC;
        B : in STD_LOGIC;
        Y : out STD_LOGIC
    );
end and_gate;

architecture Structural of and_gate is
    -- component declarations omitted: NAND_gate

    signal N1 : STD_LOGIC;
begin

    NAND1: NAND_gate
        port map (
            A => A,
            B => B,
            Y => N1
        );

    NAND2: NAND_gate
        port map (
            A => N1,
            B => N1,
            Y => Y
        );

end Structural;
```

3.4 OR gate

Both inputs are inverted by NAND gates and combined by a third NAND: Y = A OR B (De Morgan).

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity or_gate is
    Port (
        A : in STD_LOGIC;
        B : in STD_LOGIC;
        Y : out STD_LOGIC
    );
end or_gate;

architecture Structural of or_gate is
    -- component declarations omitted: NAND_gate

    signal N1 : STD_LOGIC;
    signal N2 : STD_LOGIC;
begin

    NAND1: NAND_gate
        port map (
            A => A,
            B => A,
            Y => N1
        );

    NAND2: NAND_gate
        port map (
            A => B,
            B => B,
            Y => N2
        );

    NAND3: NAND_gate
        port map (
            A => N1,
            B => N2,
            Y => Y
```

```
        );

end Structural;
```

3.5 XOR gate

Standard four-NAND XOR structure. Used in the adders and to invert B for subtraction in the ALU.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity xor_gate is
    Port (
        A : in STD_LOGIC;
        B : in STD_LOGIC;
        Y : out STD_LOGIC
    );
end xor_gate;

architecture Structural of xor_gate is
    -- component declarations omitted: NAND_gate

    signal N1 : STD_LOGIC;
    signal N2 : STD_LOGIC;
    signal N3 : STD_LOGIC;
begin

    NAND1: NAND_gate
        port map (
            A => A,
            B => B,
            Y => N1
        );

    NAND2: NAND_gate
        port map (
            A => A,
            B => N1,
            Y => N2
        );

    NAND3: NAND_gate
        port map (
            A => B,
            B => N1,
            Y => N3
        );

    NAND4: NAND_gate
        port map (
            A => N2,
            B => N3,
            Y => Y
        );

end Structural;
```

3.6 Half adder

SUM = A XOR B and CARRY = A AND B. Also used as the incrementer cell of the program counter.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity half_adder is
    Port (
        A     : in  STD_LOGIC;
        B     : in  STD_LOGIC;
        Sum   : out STD_LOGIC;
        Carry : out STD_LOGIC
    );
end half_adder;

architecture Structural of half_adder is
    -- component declarations omitted: xor_gate, and_gate

begin

    XOR1: xor_gate
        port map (
            A => A,
            B => B,
            Y => Sum
```

```
        );

    AND1: and_gate
        port map (
            A => A,
            B => B,
            Y => Carry
        );

end Structural;
```

3.7 Full adder

Two half adders and an OR gate for the carry.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity full_adder is
    Port (
        A    : in  STD_LOGIC;
        B    : in  STD_LOGIC;
        Cin  : in  STD_LOGIC;
        Sum  : out STD_LOGIC;
        Cout : out STD_LOGIC
    );
end full_adder;

architecture Structural of full_adder is
    -- component declarations omitted: half_adder, or_gate

    signal S1     : STD_LOGIC;
    signal C1     : STD_LOGIC;
    signal C2     : STD_LOGIC;

begin

    HA1: half_adder
        port map (
            A     => A,
            B     => B,
            Sum   => S1,
            Carry => C1
        );
    HA2: half_adder
        port map (
            A     => S1,
            B     => Cin,
            Sum   => Sum,
            Carry => C2
        );

    OR1: or_gate
        port map (
            A => C1,
            B => C2,
            Y => Cout
        );

end Structural;
```

3.8 8-bit ripple-carry adder

Eight full adders chained through the carry vector C. This is the core of the ALU.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity bit_8_full_adder is
    Port (
        A    : in  STD_LOGIC_VECTOR(7 downto 0);
        B    : in  STD_LOGIC_VECTOR(7 downto 0);
        Cin  : in  STD_LOGIC;
        Sum  : out STD_LOGIC_VECTOR(7 downto 0);
        Cout : out STD_LOGIC
    );
end bit_8_full_adder;

architecture Structural of bit_8_full_adder is
    -- component declarations omitted: full_adder
    signal C : STD_LOGIC_VECTOR(8 downto 0);

begin
```

```
    C(0) <= Cin;

    FA0: full_adder port map (A(0), B(0), C(0), Sum(0), C(1));
    FA1: full_adder port map (A(1), B(1), C(1), Sum(1), C(2));
    FA2: full_adder port map (A(2), B(2), C(2), Sum(2), C(3));
    FA3: full_adder port map (A(3), B(3), C(3), Sum(3), C(4));
    FA4: full_adder port map (A(4), B(4), C(4), Sum(4), C(5));
    FA5: full_adder port map (A(5), B(5), C(5), Sum(5), C(6));
    FA6: full_adder port map (A(6), B(6), C(6), Sum(6), C(7));
    FA7: full_adder port map (A(7), B(7), C(7), Sum(7), C(8));

    Cout <= C(8);

end Structural;
```

3.9 D flip-flop

Positive-edge D flip-flop with a synchronous, active-high reset: Q only changes at a rising edge of CLK, and RESET = 1 at that edge stores 0.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Positive-edge D flip-flop with SYNCHRONOUS, active-high reset
entity D_flip_flop is
    Port (
        D     : in  STD_LOGIC;
        CLK   : in  STD_LOGIC;
        RESET : in  STD_LOGIC;
        Q     : out STD_LOGIC
    );
end D_flip_flop;

architecture Behavioral of D_flip_flop is
begin

    process (CLK)
    begin
        if rising_edge(CLK) then
            if RESET = '1' then
                Q <= '0';
            else
                Q <= D;
            end if;
        end if;
    end process;

end Behavioral;
```

3.10 8-bit register

Eight D flip-flops sharing CLK and RESET. It is the storage element of the accumulator.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity register_8_bit is
    Port (
        D     : in  STD_LOGIC_VECTOR(7 downto 0);
        CLK   : in  STD_LOGIC;
        RESET : in  STD_LOGIC;
        Q     : out STD_LOGIC_VECTOR(7 downto 0)
    );
end register_8_bit;

architecture Structural of register_8_bit is
    -- component declarations omitted: D_flip_flop

begin

    FF0: D_flip_flop port map (D => D(0), CLK => CLK, RESET => RESET, Q => Q(0));
    FF1: D_flip_flop port map (D => D(1), CLK => CLK, RESET => RESET, Q => Q(1));
    FF2: D_flip_flop port map (D => D(2), CLK => CLK, RESET => RESET, Q => Q(2));
    FF3: D_flip_flop port map (D => D(3), CLK => CLK, RESET => RESET, Q => Q(3));
    FF4: D_flip_flop port map (D => D(4), CLK => CLK, RESET => RESET, Q => Q(4));
    FF5: D_flip_flop port map (D => D(5), CLK => CLK, RESET => RESET, Q => Q(5));
    FF6: D_flip_flop port map (D => D(6), CLK => CLK, RESET => RESET, Q => Q(6));
    FF7: D_flip_flop port map (D => D(7), CLK => CLK, RESET => RESET, Q => Q(7));

end Structural;
```

4. Main Components

Each main component is described below with its block diagram, VHDL code, the stimulus used in its testbench and the simulated waveform.

4.1 ALU (alu_8bit)

The ALU is combinational and supports the three arithmetic/shift operations of Set 6, chosen by SEL: 00 = ADD, 01 = SUB, 10 = SHL (11 simply passes A and is not used). Addition and subtraction share the 8-bit ripple-carry adder: SUB_MODE = SEL(0) AND NOT SEL(1) is 1 only for SUB. Every bit of B is XORed with SUB_MODE, and SUB_MODE is also the carry-in, so the adder computes A + B for ADD and A + NOT(B) + 1 = A - B for SUB. The shift left needs no gates at all: it is the wiring A(6 downto 0) & '0'. A multiplexer selects the result. COUT is the adder carry (for SUB, COUT = 1 means no borrow); it is not used by the CPU because Set 6 has no carry flag. JZ does not use the ALU.

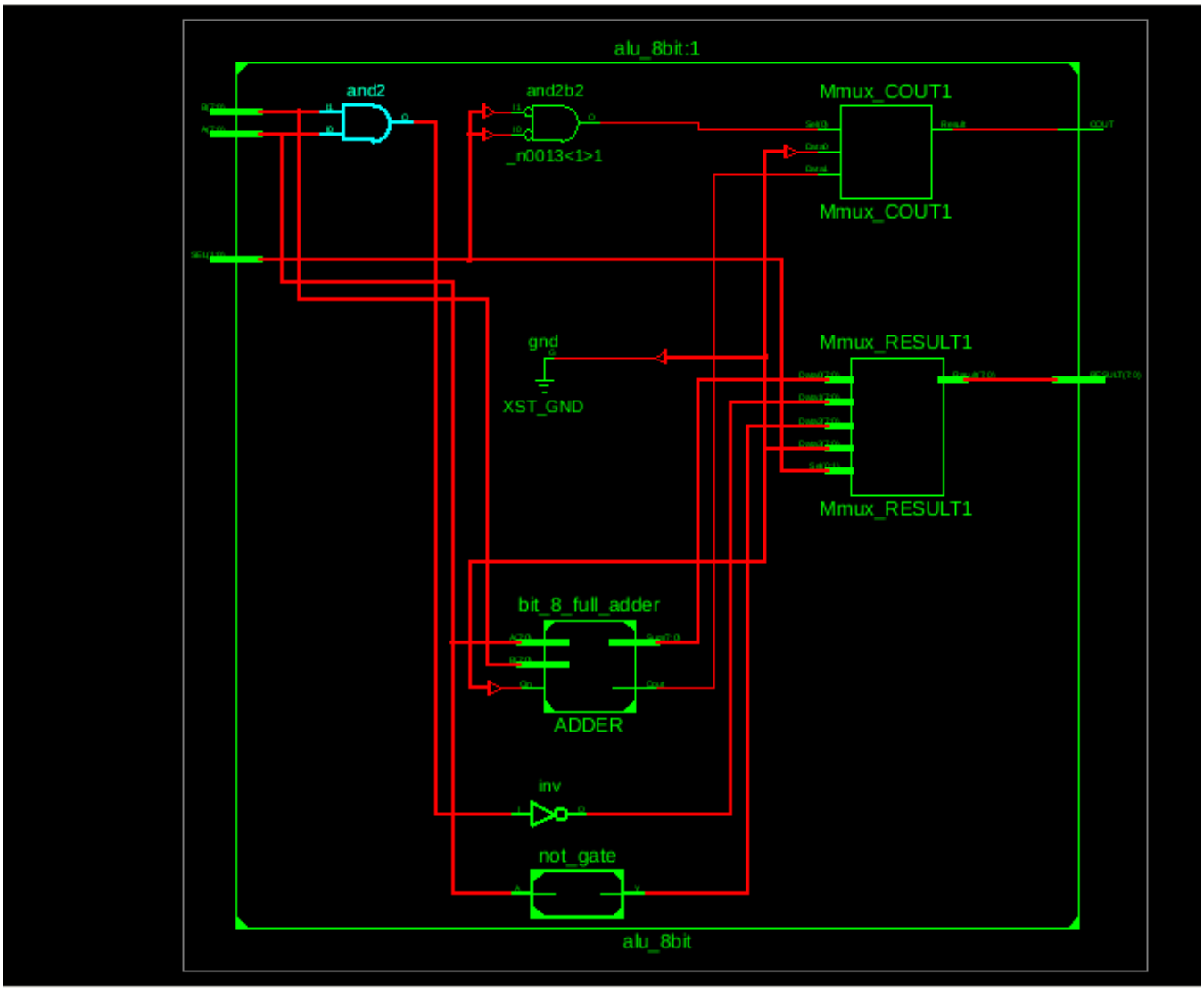


Figure 2: *Block diagram of alu_8bit*

VHDL code: alu_8bit.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- 8-bit ALU for Set 6
--   SEL = 00 : RESULT = A + B
--   SEL = 01 : RESULT = A - B   (A + NOT B + 1, modulo 256)
--   SEL = 10 : RESULT = A << 1  (0 enters the LSB, B ignored)
```

```
--   SEL = 11 : RESULT = A       (pass, not used by the CPU)
-- COUT = adder carry out for ADD / SUB (for SUB: 1 = no borrow)
entity alu_8bit is
    Port (
        A      : in  STD_LOGIC_VECTOR(7 downto 0);
        B      : in  STD_LOGIC_VECTOR(7 downto 0);
        SEL    : in  STD_LOGIC_VECTOR(1 downto 0);
        RESULT : out STD_LOGIC_VECTOR(7 downto 0);
        COUT   : out STD_LOGIC
    );
end alu_8bit;

architecture Structural of alu_8bit is
    -- component declarations omitted: bit_8_full_adder, xor_gate, and_gate, not_gate

    signal SEL1_N        : STD_LOGIC;
    signal SUB_MODE      : STD_LOGIC;
    signal B_MOD         : STD_LOGIC_VECTOR(7 downto 0);
    signal ADDSUB_RESULT : STD_LOGIC_VECTOR(7 downto 0);
    signal SHL_RESULT    : STD_LOGIC_VECTOR(7 downto 0);
    signal CARRY_OUT     : STD_LOGIC;

begin

    -- SUB_MODE = SEL(0) AND NOT SEL(1)  (only for SEL = 01)
    INV_SEL1: not_gate port map (A => SEL(1), Y => SEL1_N);
    AND_SUB : and_gate port map (A => SEL(0), B => SEL1_N, Y => SUB_MODE);

    -- B XOR SUB_MODE: passes B for ADD, inverts B for SUB
    GEN_BMOD: for i in 0 to 7 generate
        XB: xor_gate port map (A => B(i), B => SUB_MODE, Y => B_MOD(i));
    end generate GEN_BMOD;

    -- A + B (+0)  or  A + NOT B + 1
    ADDER: bit_8_full_adder
        port map (
            A    => A,
            B    => B_MOD,
            Cin  => SUB_MODE,
            Sum  => ADDSUB_RESULT,
            Cout => CARRY_OUT
        );

    -- logical shift left: wiring only, 0 enters the LSB
    SHL_RESULT <= A(6 downto 0) & '0';

    -- result multiplexer
    process (SEL, A, ADDSUB_RESULT, SHL_RESULT, CARRY_OUT)
    begin
        case SEL is
            when "00" | "01" =>
                RESULT <= ADDSUB_RESULT;
                COUT   <= CARRY_OUT;
            when "10" =>
                RESULT <= SHL_RESULT;
                COUT   <= '0';
            when others =>
                RESULT <= A;
                COUT   <= '0';
        end case;
    end process;

end Structural;
```

Testbench stimulus: alu_8bit_tb.vhd (excerpt)

```
    stim_proc: PROCESS
    BEGIN
        -- Test 1: ADD 0x42 + 0x01 = 0x43
        A <= x"42"; B <= x"01"; SEL <= "00";
        WAIT FOR 20 ns;
        ASSERT RESULT = x"43" AND COUT = '0' REPORT "Test 1 FAILED: ADD" SEVERITY ERROR;

        -- Test 2: ADD 0xAA + 0xAA = 0x54, carry out
        A <= x"AA"; B <= x"AA"; SEL <= "00";
        WAIT FOR 20 ns;
        ASSERT RESULT = x"54" AND COUT = '1' REPORT "Test 2 FAILED: ADD carry" SEVERITY ERROR;

        -- Test 3: SUB 0x02 - 0x01 = 0x01 (no borrow, COUT = 1)
        A <= x"02"; B <= x"01"; SEL <= "01";
        WAIT FOR 20 ns;
        ASSERT RESULT = x"01" AND COUT = '1' REPORT "Test 3 FAILED: SUB" SEVERITY ERROR;

        -- Test 4: SUB 0x01 - 0x01 = 0x00
        A <= x"01"; B <= x"01"; SEL <= "01";
        WAIT FOR 20 ns;
        ASSERT RESULT = x"00" AND COUT = '1' REPORT "Test 4 FAILED: SUB to zero" SEVERITY ERROR;
```

```
        -- Test 5: SUB 0x05 - 0x09 = 0xFC (borrow, COUT = 0)
        A <= x"05"; B <= x"09"; SEL <= "01";
        WAIT FOR 20 ns;
        ASSERT RESULT = x"FC" AND COUT = '0' REPORT "Test 5 FAILED: SUB borrow" SEVERITY ERROR;

        -- Test 6: SHL 0x21 = 0x42
        A <= x"21"; B <= x"00"; SEL <= "10";
        WAIT FOR 20 ns;
        ASSERT RESULT = x"42" REPORT "Test 6 FAILED: SHL" SEVERITY ERROR;

        -- Test 7: SHL 0x81 = 0x02 (MSB is lost, 0 enters LSB)
        A <= x"81"; B <= x"00"; SEL <= "10";
        WAIT FOR 20 ns;
        ASSERT RESULT = x"02" REPORT "Test 7 FAILED: SHL" SEVERITY ERROR;

        REPORT "All ALU tests completed." SEVERITY NOTE;
        WAIT;
    END PROCESS;
```

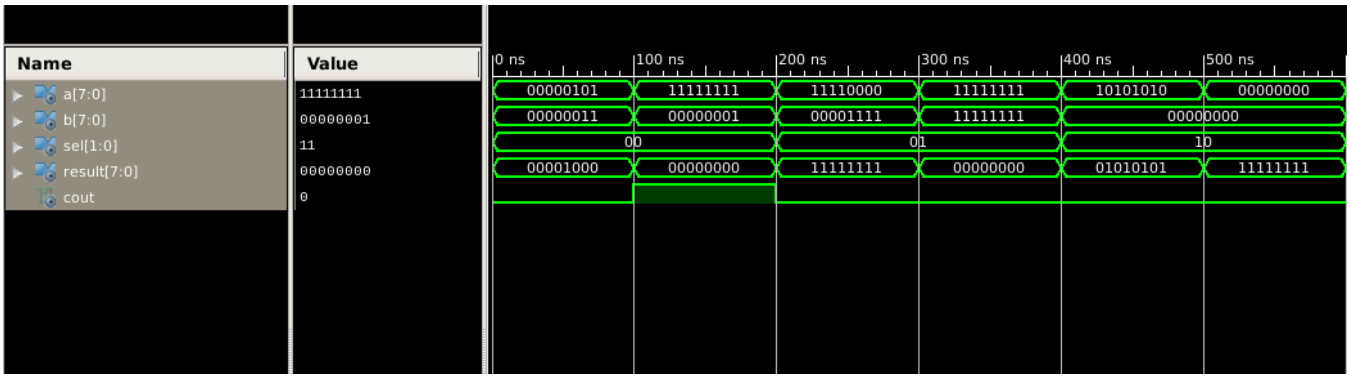


Figure 3: *Simulation waveform of alu_8bit*

ADD: 0x42 + 0x01 = 0x43 and 0xAA + 0xAA = 0x54 with COUT = 1. SUB: 0x02 - 0x01 = 0x01, 0x01 - 0x01 = 0x00 (COUT = 1, no borrow) and 0x05 - 0x09 = 0xFC (COUT = 0, borrow). SHL: 0x21 becomes 0x42 and 0x81 becomes 0x02 (the MSB is shifted out, 0 enters the LSB). All results are correct.

4.2 Zero flag (zero_flag)

The zero detector is an 8-input NOR built from a tree of seven OR gates and one NOT gate, so ZERO = 1 only when every input bit is 0. In cpu_top it watches ACC_DATA, the value that is about to be written into the accumulator, and its output is stored in a D flip-flop at the same rising edge as ACC. Because the D flip-flop resets to 0, the flip-flop stores NOT Z; the output Z = NOT(stored bit) is therefore 1 after RESET, as the specification requires. When no register is written (JZ or NOP), the flip-flop holds its value, so JZ tests the Z produced by the previous instruction.

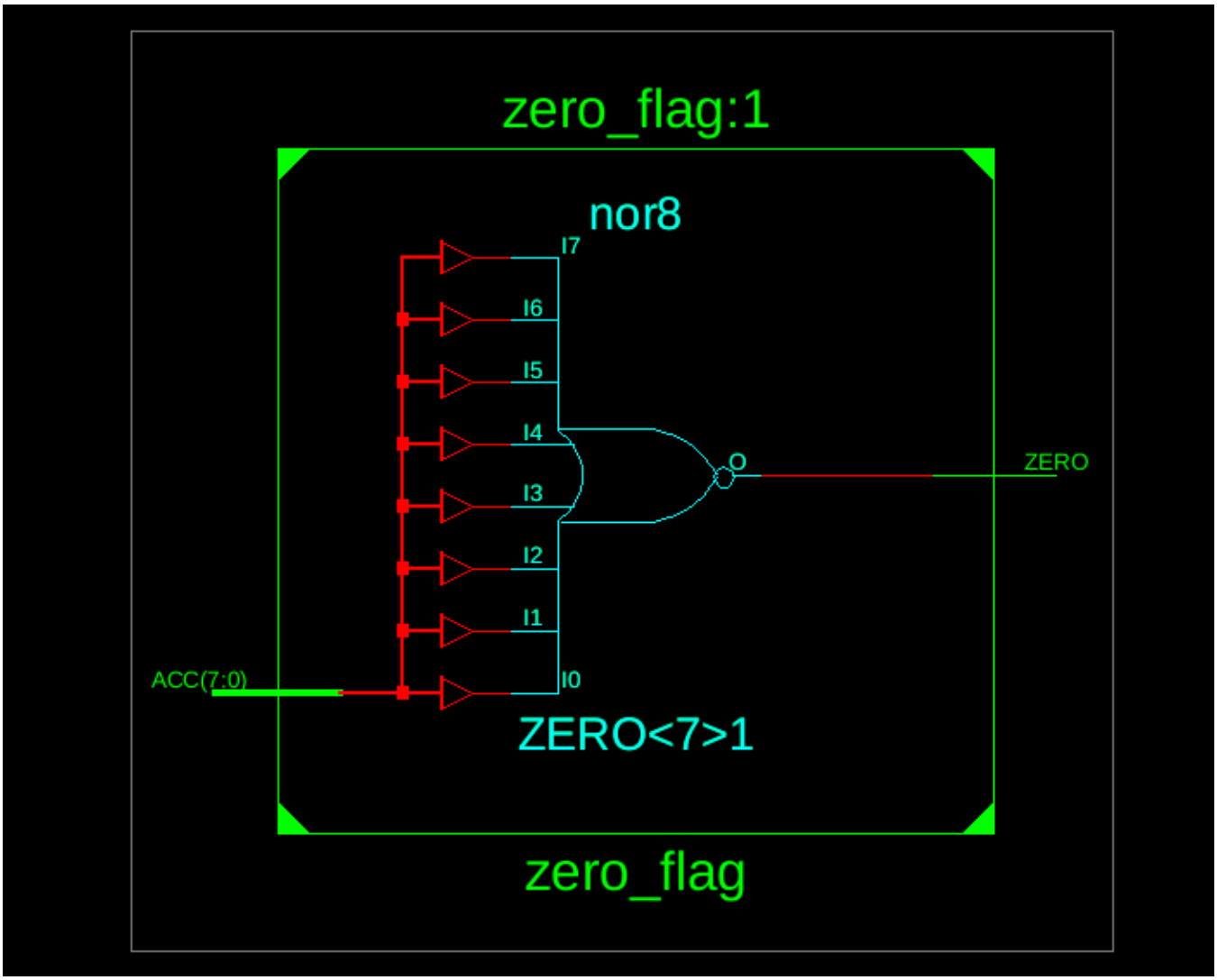


Figure 4: *RTL schematic of zero_flag*

VHDL code: zero_flag.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Zero detector: ZERO = 1 only when all 8 bits of ACC are 0.
-- Tree of seven NAND-based OR gates followed by a NOT gate (8-input NOR).
entity zero_flag is
    Port (
        ACC  : in  STD_LOGIC_VECTOR(7 downto 0);
        ZERO : out STD_LOGIC
    );
end zero_flag;

architecture Structural of zero_flag is
    -- component declarations omitted: or_gate, not_gate

    signal O1, O2, O3, O4 : STD_LOGIC;
    signal O5, O6, ANY_ONE : STD_LOGIC;

begin

    OR1: or_gate port map (A => ACC(0), B => ACC(1), Y => O1);
    OR2: or_gate port map (A => ACC(2), B => ACC(3), Y => O2);
    OR3: or_gate port map (A => ACC(4), B => ACC(5), Y => O3);
    OR4: or_gate port map (A => ACC(6), B => ACC(7), Y => O4);
    OR5: or_gate port map (A => O1, B => O2, Y => O5);
    OR6: or_gate port map (A => O3, B => O4, Y => O6);
    OR7: or_gate port map (A => O5, B => O6, Y => ANY_ONE);

    INV: not_gate port map (A => ANY_ONE, Y => ZERO);

end Structural;
```

Testbench stimulus: zero_flag_tb.vhd (excerpt)

```
    stim_proc: PROCESS
    BEGIN
        -- Test 1: all bits zero
        ACC <= x"00"; WAIT FOR 20 ns;
        ASSERT ZERO = '1' REPORT "Test 1 FAILED: expected ZERO = 1" SEVERITY ERROR;
        -- Test 2: only LSB set
        ACC <= x"01"; WAIT FOR 20 ns;
        ASSERT ZERO = '0' REPORT "Test 2 FAILED: expected ZERO = 0" SEVERITY ERROR;
        -- Test 3: only MSB set
        ACC <= x"80"; WAIT FOR 20 ns;
        ASSERT ZERO = '0' REPORT "Test 3 FAILED: expected ZERO = 0" SEVERITY ERROR;
        -- Test 4: all bits set
        ACC <= x"FF"; WAIT FOR 20 ns;
        ASSERT ZERO = '0' REPORT "Test 4 FAILED: expected ZERO = 0" SEVERITY ERROR;
        -- Test 5: zero again
        ACC <= x"00"; WAIT FOR 20 ns;
        ASSERT ZERO = '1' REPORT "Test 5 FAILED: expected ZERO = 1" SEVERITY ERROR;

        REPORT "All zero flag tests completed." SEVERITY NOTE;
        WAIT;
    END PROCESS;
```

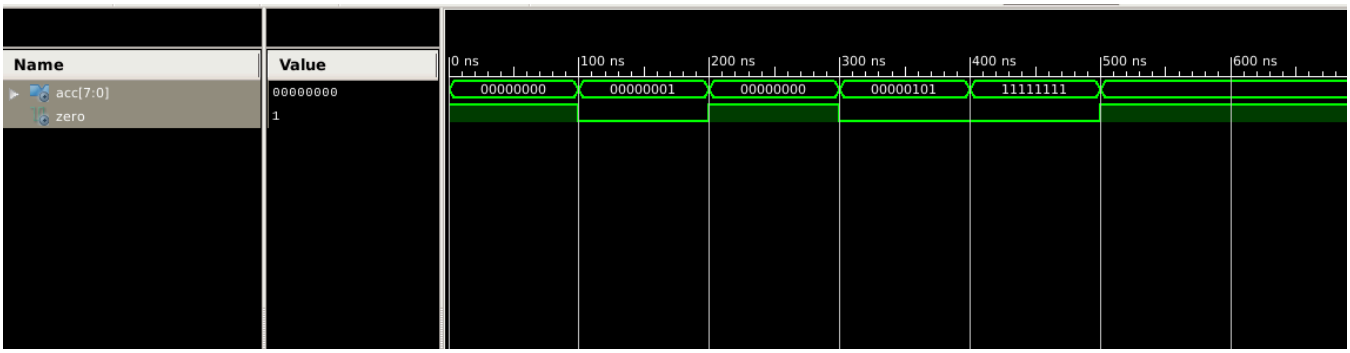


Figure 5: *Simulation waveform of zero_flag*

ZERO is 1 for 0x00, drops to 0 for 0x01, 0x80 and 0xFF (single LSB, single MSB and all bits set), and returns to 1 when the input is 0x00 again.

4.3 Accumulator (accumulator_8bit)

The accumulator is the main working register and the CPU output Q. It consists of the 8-bit register and a hold/load multiplexer. With LOAD = 0 the register input is its own output, so the value is held; with LOAD = 1 it loads DATA_IN at the next rising clock edge. RESET is synchronous, active-high and has priority over LOAD. In cpu_top, LOAD is driven by ACC_WE from the control unit.

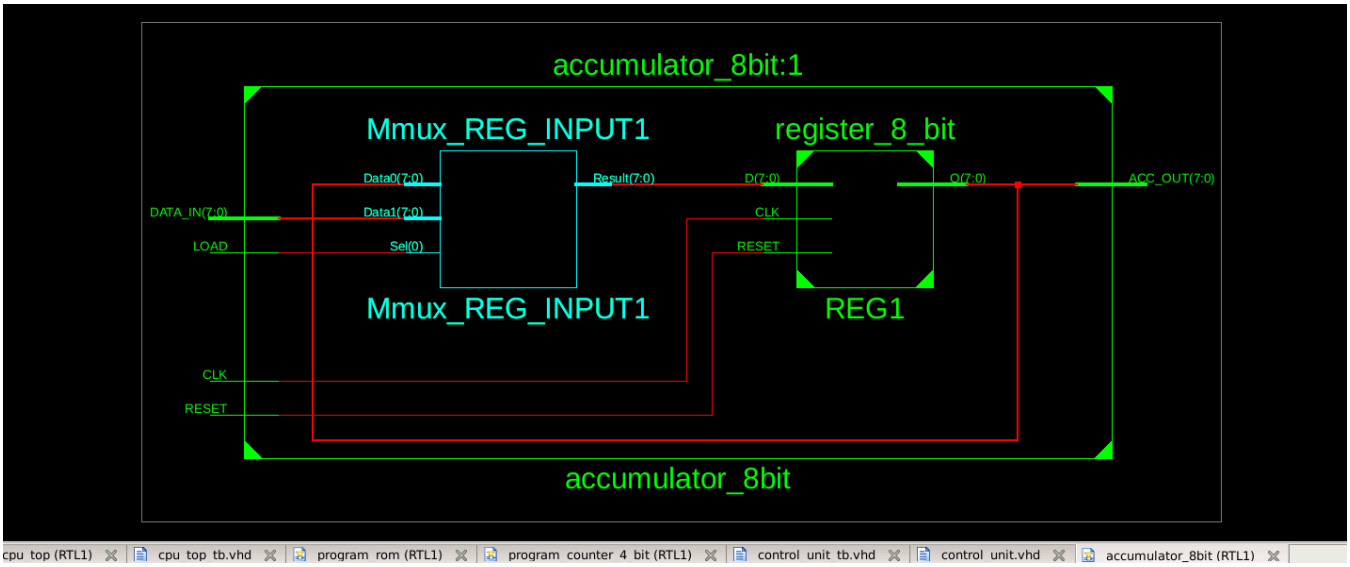


Figure 6: *RTL schematic of accumulator_8bit*

VHDL code: accumulator_8bit.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity accumulator_8bit is
    Port (
        CLK     : in  STD_LOGIC;
        RESET   : in  STD_LOGIC;
        DATA_IN : in  STD_LOGIC_VECTOR(7 downto 0);
        LOAD    : in  STD_LOGIC;
        ACC_OUT : out STD_LOGIC_VECTOR(7 downto 0)
    );
end accumulator_8bit;

architecture Structural of accumulator_8bit is
    -- component declarations omitted: register_8_bit

    signal REG_INPUT : STD_LOGIC_VECTOR(7 downto 0);
    signal REG_OUTPUT : STD_LOGIC_VECTOR(7 downto 0);

begin

    -- Input selection logic
    process (DATA_IN, REG_OUTPUT, LOAD)
    begin
        if LOAD = '1' then
            REG_INPUT <= DATA_IN;
        else
            REG_INPUT <= REG_OUTPUT;
        end if;
    end process;

    -- 8-bit register
    REG1: register_8_bit
        port map (
            D     => REG_INPUT,
            CLK   => CLK,
            RESET => RESET,
            Q     => REG_OUTPUT
        );

    -- Accumulator output
    ACC_OUT <= REG_OUTPUT;

end Structural;
```

Testbench stimulus: accumulator_8bit_tb.vhd (excerpt)

```
    stim_proc: PROCESS
    BEGIN
        -- Test 1: reset
        RESET <= '1'; LOAD <= '0'; DATA_IN <= x"00";
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
        ASSERT ACC_OUT = x"00" REPORT "Test 1 FAILED: reset" SEVERITY ERROR;

        -- Test 2: load 0x5A
        RESET <= '0'; LOAD <= '1'; DATA_IN <= x"5A";
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
        ASSERT ACC_OUT = x"5A" REPORT "Test 2 FAILED: load 5A" SEVERITY ERROR;

        -- Test 3: hold (LOAD = 0), input changes to 0xFF
        LOAD <= '0'; DATA_IN <= x"FF";
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
        ASSERT ACC_OUT = x"5A" REPORT "Test 3 FAILED: hold" SEVERITY ERROR;

        -- Test 4: load 0xC3
        LOAD <= '1'; DATA_IN <= x"C3";
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
        ASSERT ACC_OUT = x"C3" REPORT "Test 4 FAILED: load C3" SEVERITY ERROR;

        -- Test 5: reset has priority over LOAD
        RESET <= '1'; LOAD <= '1'; DATA_IN <= x"77";
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
        ASSERT ACC_OUT = x"00" REPORT "Test 5 FAILED: reset priority" SEVERITY ERROR;

        RESET <= '0'; LOAD <= '0';
        REPORT "All accumulator tests completed." SEVERITY NOTE;
        WAIT;
    END PROCESS;
```

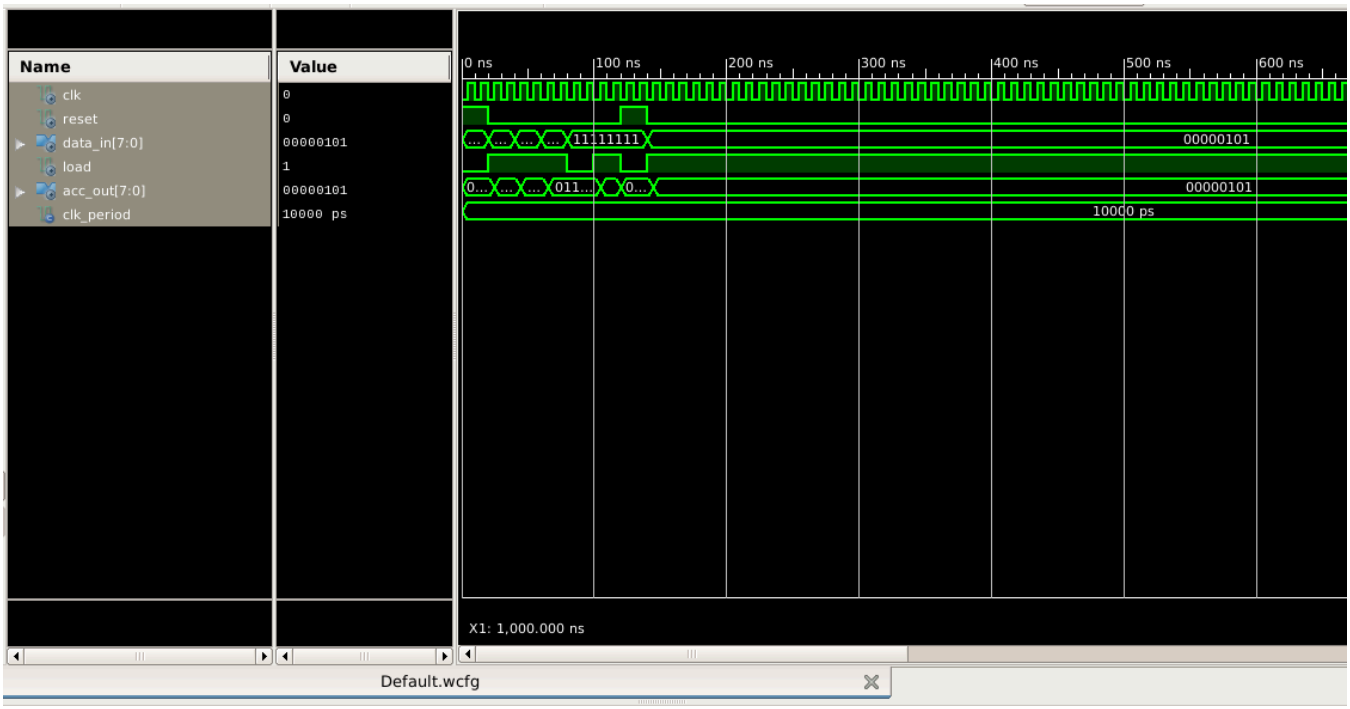


Figure 7: *Simulation waveform of accumulator_8bit*

After reset ACC_OUT = 0x00. At the next edge with LOAD = 1 the value 0x5A is loaded. When DATA_IN changes to 0xFF while LOAD = 0, ACC_OUT stays 0x5A (hold). With LOAD = 1 again 0xC3 is loaded, and the final RESET clears the register to 0x00 even though LOAD = 1.

4.4 Program counter (program_counter_4_bit)

The program counter is a 4-bit register built from four D flip-flops, with an incrementer made of four half adders in a ripple chain (the first carry-in is the constant 1). A 2-to-1 multiplexer in front of each flip-flop chooses the next value: the jump target D when LOAD = 1, otherwise PC + 1. In cpu_top, LOAD is PC_LOAD (JZ taken) and D is operand[3:0]. A rising edge with RESET = 1 sets PC to 0 and has priority over LOAD.

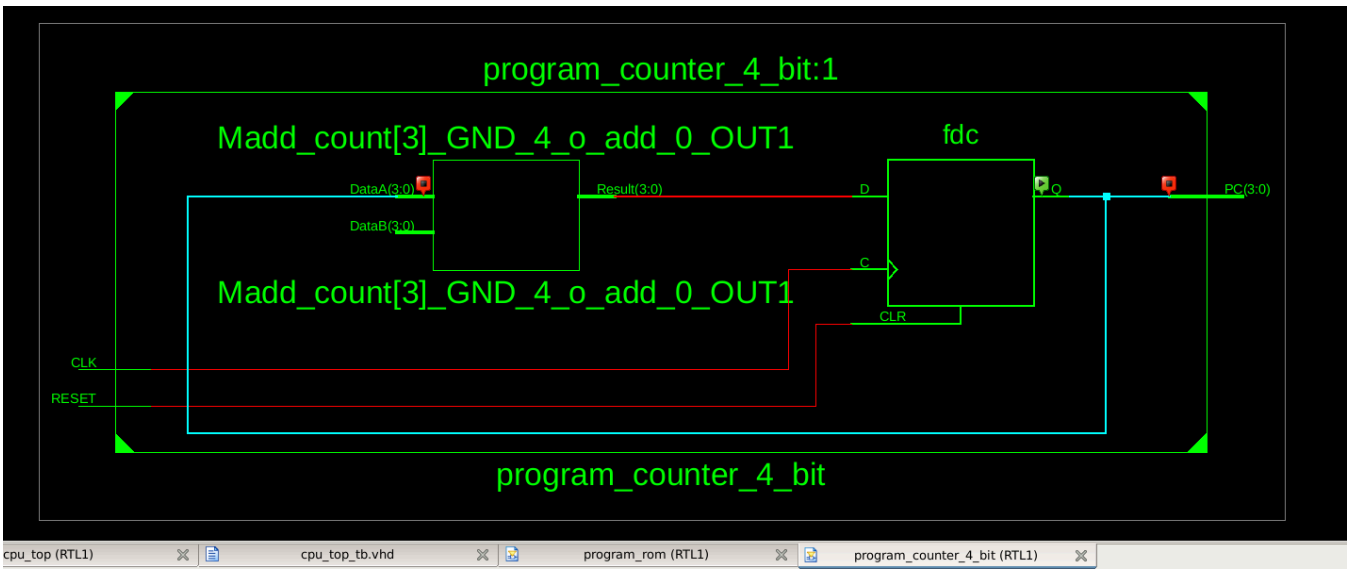


Figure 8: *Block diagram of program_counter_4_bit*

VHDL code: program_counter_4_bit.vhd

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;

-- 4-bit program counter with load (for JZ)
--   RESET = 1 at a rising edge : PC <- 0        (synchronous)
--   LOAD  = 1 at a rising edge : PC <- D        (jump target)
--   otherwise                  : PC <- PC + 1
-- Incrementer = chain of 4 half adders, storage = 4 D flip-flops.
entity program_counter_4_bit is
    Port (
        CLK   : in  STD_LOGIC;
        RESET : in  STD_LOGIC;
        LOAD  : in  STD_LOGIC;
        D     : in  STD_LOGIC_VECTOR(3 downto 0);
        PC    : out STD_LOGIC_VECTOR(3 downto 0)
    );
end program_counter_4_bit;

architecture Structural of program_counter_4_bit is
    -- component declarations omitted: half_adder, D_flip_flop

    signal PC_Q    : STD_LOGIC_VECTOR(3 downto 0);
    signal PC_INC  : STD_LOGIC_VECTOR(3 downto 0);
    signal PC_NEXT : STD_LOGIC_VECTOR(3 downto 0);
    signal C       : STD_LOGIC_VECTOR(4 downto 0);

begin

    C(0) <= '1';   -- adding 1

    GEN_PC: for i in 0 to 3 generate
        -- PC + 1
        INC: half_adder
            port map (
                A     => PC_Q(i),
                B     => C(i),
                Sum   => PC_INC(i),
                Carry => C(i+1)
            );

        -- next-PC multiplexer: jump target or PC + 1
        PC_NEXT(i) <= D(i) when LOAD = '1' else PC_INC(i);

        FF: D_flip_flop
            port map (
                D     => PC_NEXT(i),
                CLK   => CLK,
                RESET => RESET,
                Q     => PC_Q(i)
            );
    end generate GEN_PC;

    PC <= PC_Q;

end Structural;
```

Testbench stimulus: program_counter_4_bit_tb.vhd (excerpt)

```
    stim_proc: PROCESS
    BEGIN
        -- Test 1: synchronous reset
        RESET <= '1'; LOAD <= '0';
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
        ASSERT PC = "0000" REPORT "Test 1 FAILED: reset" SEVERITY ERROR;

        -- Test 2: count 1, 2, 3
        RESET <= '0';
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
        ASSERT PC = "0001" REPORT "Test 2 FAILED: PC should be 1" SEVERITY ERROR;
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
        ASSERT PC = "0010" REPORT "Test 2 FAILED: PC should be 2" SEVERITY ERROR;
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
        ASSERT PC = "0011" REPORT "Test 2 FAILED: PC should be 3" SEVERITY ERROR;

        -- Test 3: load jump address 7
        D <= "0111"; LOAD <= '1';
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
        ASSERT PC = "0111" REPORT "Test 3 FAILED: PC should load 7" SEVERITY ERROR;

        -- Test 4: increment after the jump
        LOAD <= '0';
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
        ASSERT PC = "1000" REPORT "Test 4 FAILED: PC should be 8" SEVERITY ERROR;

        -- Test 5: wrap around 15 -> 0
        D <= "1111"; LOAD <= '1';
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
        LOAD <= '0';
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
```

```
        ASSERT PC = "0000" REPORT "Test 5 FAILED: wrap around" SEVERITY ERROR;

        -- Test 6: reset has priority over LOAD
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
        RESET <= '1'; LOAD <= '1'; D <= "0101";
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
        ASSERT PC = "0000" REPORT "Test 6 FAILED: reset priority" SEVERITY ERROR;
        RESET <= '0'; LOAD <= '0';

        REPORT "All program counter tests completed." SEVERITY NOTE;
        WAIT;
    END PROCESS;
```

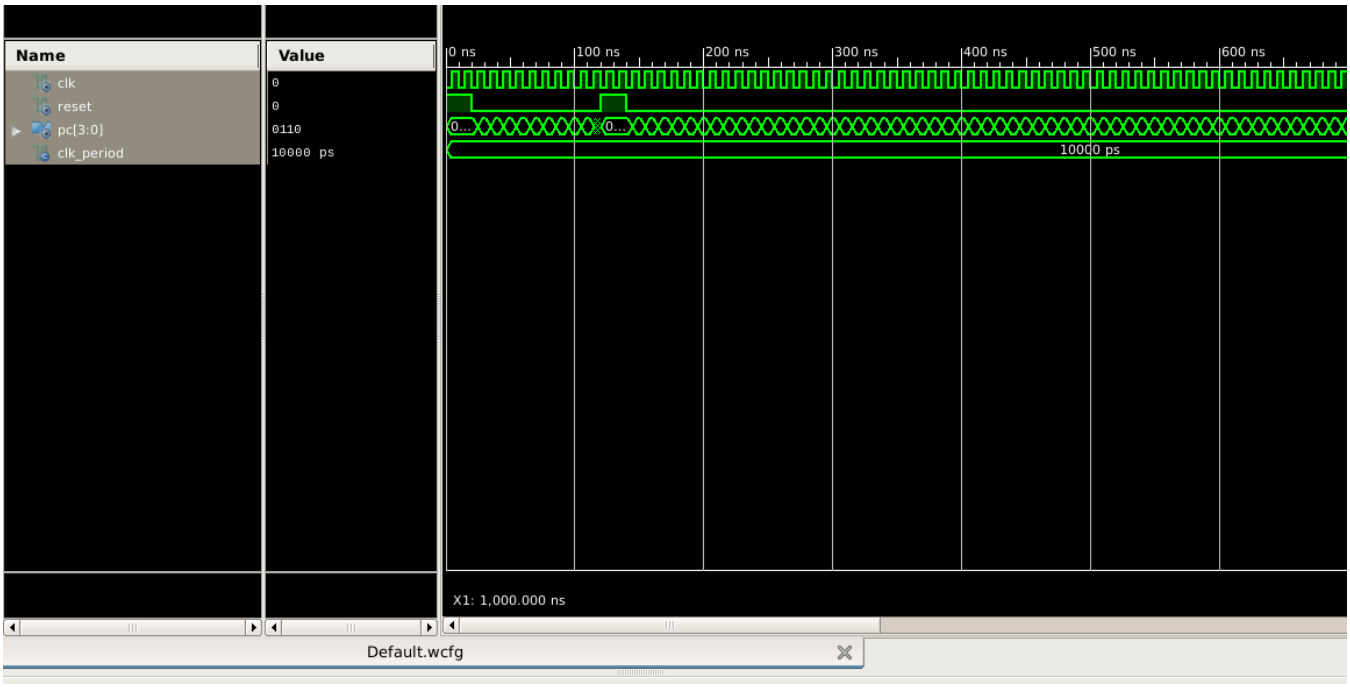


Figure 9: *Simulation waveform of program_counter_4_bit*

PC is 0 after the reset edge and counts 1, 2, 3. With LOAD = 1 and D = 7 it jumps to 7 and then continues with 8. Loading 15 and counting once shows the wrap-around to 0. Finally RESET and LOAD are both 1 with D = 5, and PC becomes 0, so the reset has priority.

4.5 Program ROM (program_rom)

The ROM is a 16 x 12-bit memory written as a VHDL case statement. It is read combinationally with PC as the address (no clock). It contains the ten instructions of Table 3; every other address returns 0x000 (NOP).

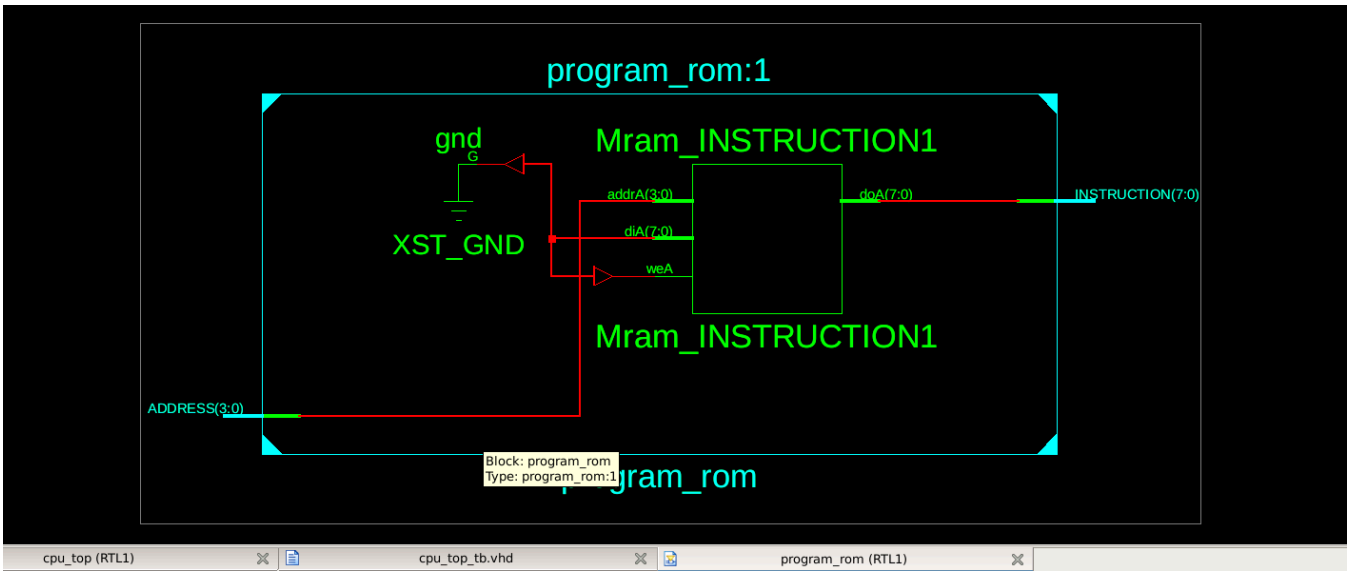


Figure 10: *Block symbol of program_rom*

VHDL code: program_rom.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- 16 x 12-bit program ROM (combinational read)
-- instr(11:8) = opcode, instr(7:0) = operand
-- Holds the Appendix B program of Set 6.
entity program_rom is
    Port (
        ADDRESS     : in  STD_LOGIC_VECTOR(3 downto 0);
        INSTRUCTION : out STD_LOGIC_VECTOR(11 downto 0)
    );
end program_rom;

architecture Behavioral of program_rom is
begin

    process (ADDRESS)
    begin
        case ADDRESS is
            when "0000" => INSTRUCTION <= x"102"; -- LDI 0x02
            when "0001" => INSTRUCTION <= x"301"; -- SUB 0x01
            when "0010" => INSTRUCTION <= x"C07"; -- JZ  0x07
            when "0011" => INSTRUCTION <= x"301"; -- SUB 0x01
            when "0100" => INSTRUCTION <= x"C07"; -- JZ  0x07
            when "0101" => INSTRUCTION <= x"2AA"; -- ADD 0xAA
            when "0110" => INSTRUCTION <= x"2AA"; -- ADD 0xAA
            when "0111" => INSTRUCTION <= x"121"; -- LDI 0x21
            when "1000" => INSTRUCTION <= x"800"; -- SHL
            when "1001" => INSTRUCTION <= x"201"; -- ADD 0x01
            when others => INSTRUCTION <= x"000"; -- NOP
        end case;
    end process;

end Behavioral;
```

Testbench stimulus: program_rom_tb.vhd (excerpt)

```
    stim_proc: PROCESS
    BEGIN
        ADDRESS <= x"0"; WAIT FOR 10 ns;
        ASSERT INSTRUCTION = x"102" REPORT "Address 0 failed" SEVERITY ERROR;
        ADDRESS <= x"1"; WAIT FOR 10 ns;
        ASSERT INSTRUCTION = x"301" REPORT "Address 1 failed" SEVERITY ERROR;
        ADDRESS <= x"2"; WAIT FOR 10 ns;
        ASSERT INSTRUCTION = x"C07" REPORT "Address 2 failed" SEVERITY ERROR;
        ADDRESS <= x"3"; WAIT FOR 10 ns;
        ASSERT INSTRUCTION = x"301" REPORT "Address 3 failed" SEVERITY ERROR;
        ADDRESS <= x"4"; WAIT FOR 10 ns;
        ASSERT INSTRUCTION = x"C07" REPORT "Address 4 failed" SEVERITY ERROR;
        ADDRESS <= x"5"; WAIT FOR 10 ns;
        ASSERT INSTRUCTION = x"2AA" REPORT "Address 5 failed" SEVERITY ERROR;
        ADDRESS <= x"6"; WAIT FOR 10 ns;
        ASSERT INSTRUCTION = x"2AA" REPORT "Address 6 failed" SEVERITY ERROR;
        ADDRESS <= x"7"; WAIT FOR 10 ns;
        ASSERT INSTRUCTION = x"121" REPORT "Address 7 failed" SEVERITY ERROR;
        ADDRESS <= x"8"; WAIT FOR 10 ns;
```

```
        ASSERT INSTRUCTION = x"800" REPORT "Address 8 failed" SEVERITY ERROR;
        ADDRESS <= x"9"; WAIT FOR 10 ns;
        ASSERT INSTRUCTION = x"201" REPORT "Address 9 failed" SEVERITY ERROR;
        ADDRESS <= x"A"; WAIT FOR 10 ns;
        ASSERT INSTRUCTION = x"000" REPORT "Unused address should return NOP" SEVERITY ERROR;

        REPORT "ROM tests completed." SEVERITY NOTE;
        WAIT;
    END PROCESS;
```

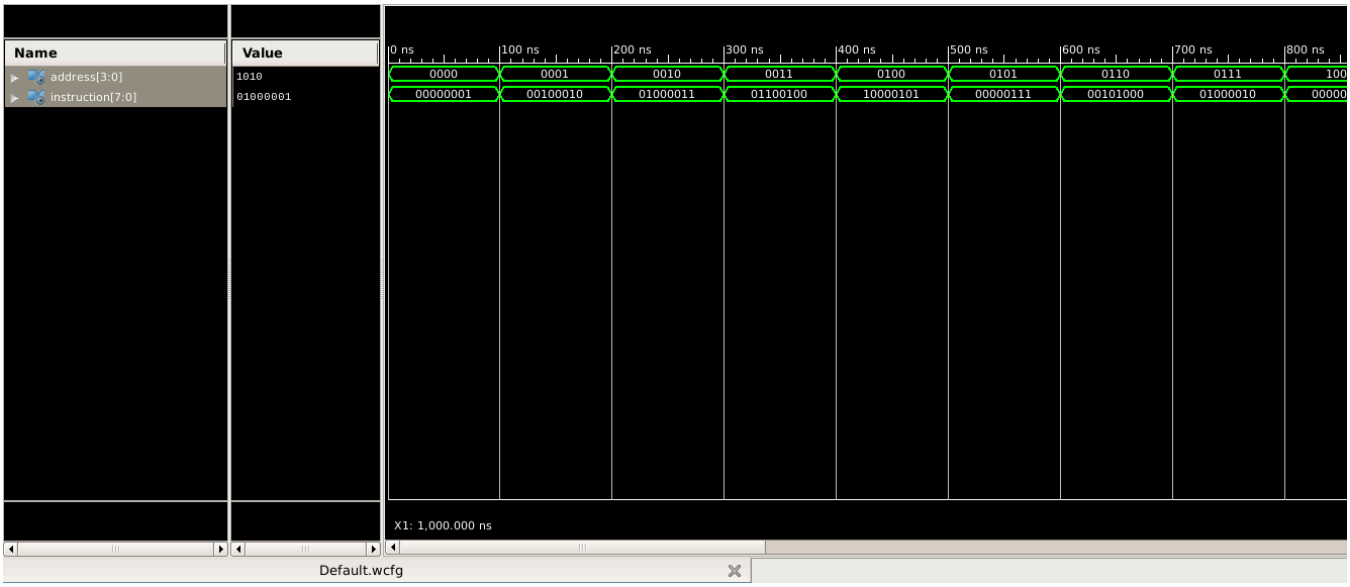


Figure 11: *Simulation waveform of program_rom*

Stepping the address gives 0x102, 0x301, 0xC07, 0x301, 0xC07, 0x2AA, 0x2AA, 0x121, 0x800 and 0x201 for addresses 0 to 9, and 0x000 for the unused address 10. The output changes immediately with the address, which confirms the combinational read.

4.6 Instruction decoder (instruction_decoder)

The decoder splits the 12-bit instruction into OPCODE (bits 11:8) and OPERAND (bits 7:0) and raises exactly one enable line for the five supported opcodes: LOAD_EN (0001), ADD_EN (0010), SUB_EN (0011), JZ_EN (1100) or SHL_EN (1000). Any other opcode, including 0000, raises no enable and behaves as a NOP.

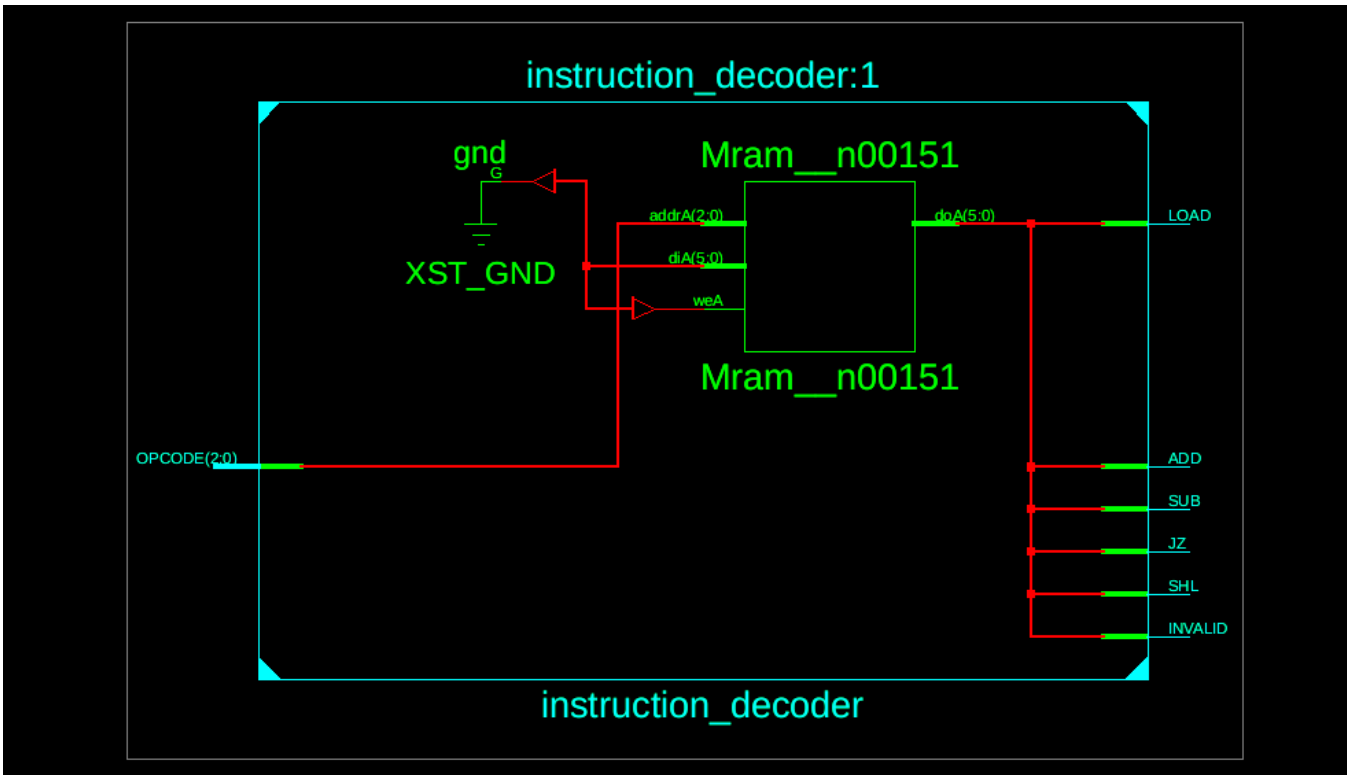


Figure 12: *Block symbol of instruction_decoder*

VHDL code: instruction_decoder.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Splits the 12-bit instruction into opcode / operand and raises
-- one enable line for each instruction of Set 6.
-- Any other opcode (including 0000) raises nothing = NOP.
entity instruction_decoder is
    Port (
        INSTRUCTION : in  STD_LOGIC_VECTOR(11 downto 0);
        OPCODE      : out STD_LOGIC_VECTOR(3 downto 0);
        OPERAND     : out STD_LOGIC_VECTOR(7 downto 0);
        LOAD_EN     : out STD_LOGIC;
        ADD_EN      : out STD_LOGIC;
        SUB_EN      : out STD_LOGIC;
        JZ_EN       : out STD_LOGIC;
        SHL_EN      : out STD_LOGIC
    );
end instruction_decoder;

architecture Behavioral of instruction_decoder is
begin

    OPCODE  <= INSTRUCTION(11 downto 8);
    OPERAND <= INSTRUCTION(7 downto 0);

    process (INSTRUCTION)
    begin
        -- default: no operation
        LOAD_EN <= '0';
        ADD_EN  <= '0';
        SUB_EN  <= '0';
        JZ_EN   <= '0';
        SHL_EN  <= '0';

        case INSTRUCTION(11 downto 8) is
            when "0001" => LOAD_EN <= '1';   -- LDI imm
            when "0010" => ADD_EN  <= '1';   -- ADD imm
            when "0011" => SUB_EN  <= '1';   -- SUB imm
            when "1100" => JZ_EN   <= '1';   -- JZ addr
            when "1000" => SHL_EN  <= '1';   -- SHL
            when others => null;             -- NOP
        end case;
    end process;

end Behavioral;
```

Testbench stimulus: instruction_decoder_tb.vhd (excerpt)

```
    stim_proc: PROCESS
    BEGIN
        -- LDI 0x02
        INSTRUCTION <= x"102"; WAIT FOR 10 ns;
        ASSERT LOAD_EN = '1' AND OPCODE = "0001" AND OPERAND = x"02"
            REPORT "LDI decode failed" SEVERITY ERROR;
        -- ADD 0xAA
        INSTRUCTION <= x"2AA"; WAIT FOR 10 ns;
        ASSERT ADD_EN = '1' AND LOAD_EN = '0' AND OPERAND = x"AA"
            REPORT "ADD decode failed" SEVERITY ERROR;
        -- SUB 0x01
        INSTRUCTION <= x"301"; WAIT FOR 10 ns;
        ASSERT SUB_EN = '1' AND ADD_EN = '0' AND OPERAND = x"01"
            REPORT "SUB decode failed" SEVERITY ERROR;
        -- JZ 0x07
        INSTRUCTION <= x"C07"; WAIT FOR 10 ns;
        ASSERT JZ_EN = '1' AND SUB_EN = '0' AND OPERAND = x"07"
            REPORT "JZ decode failed" SEVERITY ERROR;
        -- SHL
        INSTRUCTION <= x"800"; WAIT FOR 10 ns;
        ASSERT SHL_EN = '1' AND JZ_EN = '0'
            REPORT "SHL decode failed" SEVERITY ERROR;
        -- NOP (0x000) and an opcode outside Set 6 (XOR, 0x63F)
        INSTRUCTION <= x"000"; WAIT FOR 10 ns;
        ASSERT LOAD_EN = '0' AND ADD_EN = '0' AND SUB_EN = '0' AND JZ_EN = '0' AND SHL_EN = '0'
            REPORT "NOP decode failed" SEVERITY ERROR;
        INSTRUCTION <= x"63F"; WAIT FOR 10 ns;
        ASSERT LOAD_EN = '0' AND ADD_EN = '0' AND SUB_EN = '0' AND JZ_EN = '0' AND SHL_EN = '0'
            REPORT "Opcode outside Set 6 must be NOP" SEVERITY ERROR;

        REPORT "Instruction decoder tests completed." SEVERITY NOTE;
        WAIT;
    END PROCESS;
```

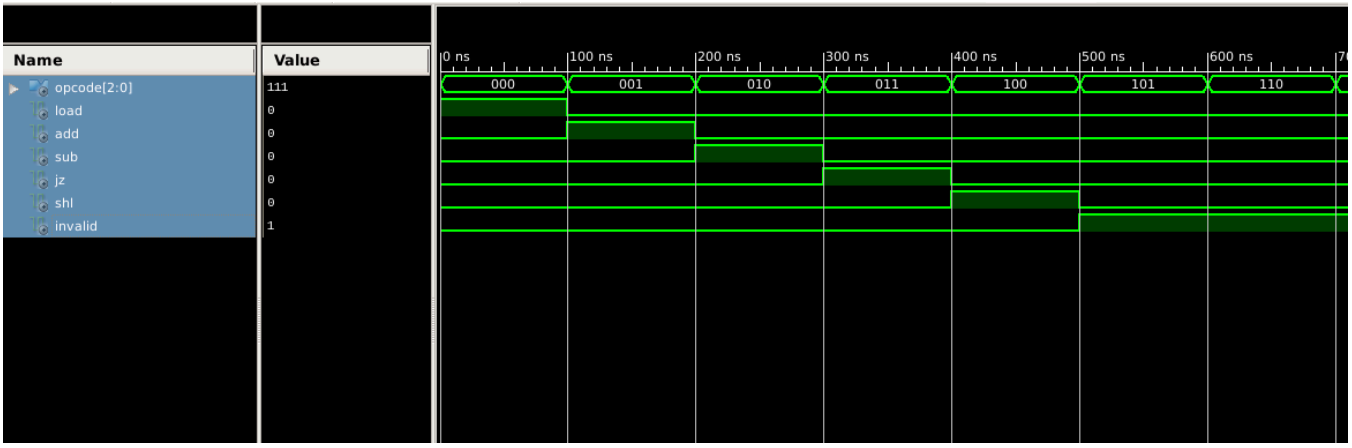


Figure 13: *Simulation waveform of instruction_decoder*

0x102 raises LOAD_EN with OPERAND = 0x02, 0x2AA raises ADD_EN, 0x301 raises SUB_EN, 0xC07 raises JZ_EN with OPERAND = 0x07 and 0x800 raises SHL_EN. For 0x000 and for 0x63F (XOR, which is not part of Set 6) all enables stay low.

4.7 Control unit (control_unit)

The control unit contains the instruction decoder and turns its enables into the datapath control signals: ALU_SEL(1) = SHL_EN and ALU_SEL(0) = SUB_EN give the ALU codes 00 (ADD), 01 (SUB) and 10 (SHL); ACC_SRC = NOT LOAD_EN selects the operand for LDI and the ALU result otherwise; ACC_WE = LOAD_EN OR ADD_EN OR SUB_EN OR SHL_EN, so JZ and NOP leave ACC and Z unchanged; and PC_LOAD = JZ_EN AND Z makes the jump happen only when the zero flag is set. The logic is combinational, so every instruction, including a taken jump, finishes in one clock cycle.

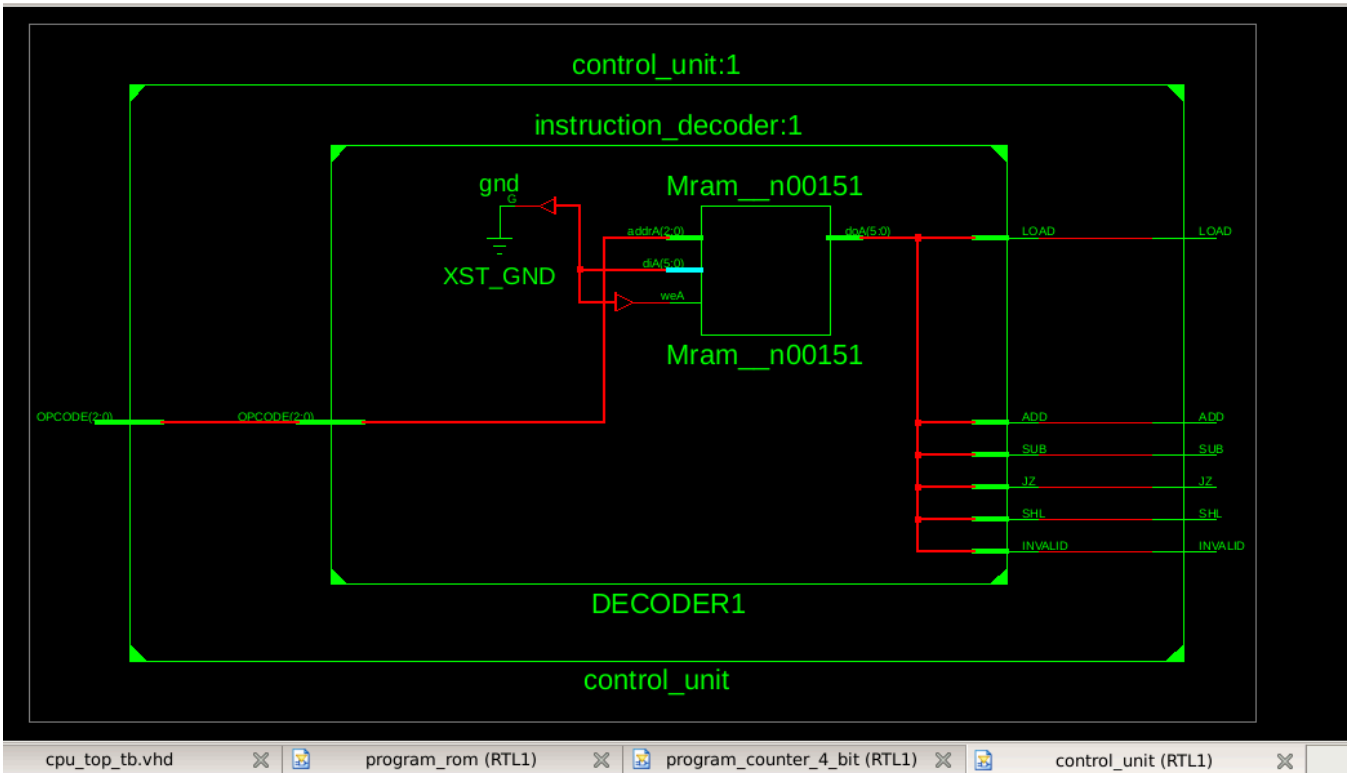


Figure 14: *Block diagram of control_unit*

VHDL code: control_unit.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Control unit: decodes the instruction and drives the datapath.
--   ALU_SEL : 00 = ADD, 01 = SUB, 10 = SHL
--   ACC_SRC : 0 = operand (LDI), 1 = ALU result
--   ACC_WE  : 1 = write ACC and Z at the next rising edge
--   PC_LOAD : 1 = load the JZ target into PC (JZ and Z = 1)
entity control_unit is
    Port (
        INSTRUCTION : in  STD_LOGIC_VECTOR(11 downto 0);
        Z           : in  STD_LOGIC;
        OPERAND     : out STD_LOGIC_VECTOR(7 downto 0);
        ALU_SEL     : out STD_LOGIC_VECTOR(1 downto 0);
        ACC_SRC     : out STD_LOGIC;
        ACC_WE      : out STD_LOGIC;
        PC_LOAD     : out STD_LOGIC;
        LOAD_EN     : out STD_LOGIC;
        ADD_EN      : out STD_LOGIC;
        SUB_EN      : out STD_LOGIC;
        JZ_EN       : out STD_LOGIC;
        SHL_EN      : out STD_LOGIC
    );
end control_unit;

architecture Structural of control_unit is
    -- component declarations omitted: instruction_decoder

    signal OPCODE_SIG : STD_LOGIC_VECTOR(3 downto 0);
    signal LOAD_SIG   : STD_LOGIC;
    signal ADD_SIG    : STD_LOGIC;
    signal SUB_SIG    : STD_LOGIC;
    signal JZ_SIG     : STD_LOGIC;
    signal SHL_SIG    : STD_LOGIC;

begin

    DECODER1: instruction_decoder
        port map (
            INSTRUCTION => INSTRUCTION,
            OPCODE      => OPCODE_SIG,
            OPERAND     => OPERAND,
            LOAD_EN     => LOAD_SIG,
            ADD_EN      => ADD_SIG,
            SUB_EN      => SUB_SIG,
            JZ_EN       => JZ_SIG,
```

```
            SHL_EN      => SHL_SIG
        );

    -- ALU function select
    ALU_SEL(1) <= SHL_SIG;
    ALU_SEL(0) <= SUB_SIG;

    -- ACC source: operand for LDI, ALU result otherwise
    ACC_SRC <= not LOAD_SIG;

    -- LDI, ADD, SUB and SHL write ACC; JZ and NOP do not
    ACC_WE <= LOAD_SIG or ADD_SIG or SUB_SIG or SHL_SIG;

    -- JZ is taken only when the zero flag is set
    PC_LOAD <= JZ_SIG and Z;

    LOAD_EN <= LOAD_SIG;
    ADD_EN  <= ADD_SIG;
    SUB_EN  <= SUB_SIG;
    JZ_EN   <= JZ_SIG;
    SHL_EN  <= SHL_SIG;

end Structural;
```

Testbench stimulus: control_unit_tb.vhd (excerpt)

```
    stim_proc: PROCESS
    BEGIN
        -- LDI: operand -> ACC
        INSTRUCTION <= x"102"; Z <= '0'; WAIT FOR 10 ns;
        ASSERT ACC_WE = '1' AND ACC_SRC = '0' AND PC_LOAD = '0' AND OPERAND = x"02"
            REPORT "LDI control failed" SEVERITY ERROR;
        -- ADD: ALU_SEL = 00
        INSTRUCTION <= x"2AA"; WAIT FOR 10 ns;
        ASSERT ACC_WE = '1' AND ACC_SRC = '1' AND ALU_SEL = "00"
            REPORT "ADD control failed" SEVERITY ERROR;
        -- SUB: ALU_SEL = 01
        INSTRUCTION <= x"301"; WAIT FOR 10 ns;
        ASSERT ACC_WE = '1' AND ACC_SRC = '1' AND ALU_SEL = "01"
            REPORT "SUB control failed" SEVERITY ERROR;
        -- SHL: ALU_SEL = 10
        INSTRUCTION <= x"800"; WAIT FOR 10 ns;
        ASSERT ACC_WE = '1' AND ACC_SRC = '1' AND ALU_SEL = "10"
            REPORT "SHL control failed" SEVERITY ERROR;
        -- JZ with Z = 0: not taken, no write
        INSTRUCTION <= x"C07"; Z <= '0'; WAIT FOR 10 ns;
        ASSERT ACC_WE = '0' AND PC_LOAD = '0'
            REPORT "JZ not-taken control failed" SEVERITY ERROR;
        -- JZ with Z = 1: taken
        Z <= '1'; WAIT FOR 10 ns;
        ASSERT ACC_WE = '0' AND PC_LOAD = '1'
            REPORT "JZ taken control failed" SEVERITY ERROR;
        -- NOP: nothing happens even with Z = 1
        INSTRUCTION <= x"000"; WAIT FOR 10 ns;
        ASSERT ACC_WE = '0' AND PC_LOAD = '0'
            REPORT "NOP control failed" SEVERITY ERROR;

        REPORT "Control unit tests completed." SEVERITY NOTE;
        WAIT;
    END PROCESS;
```

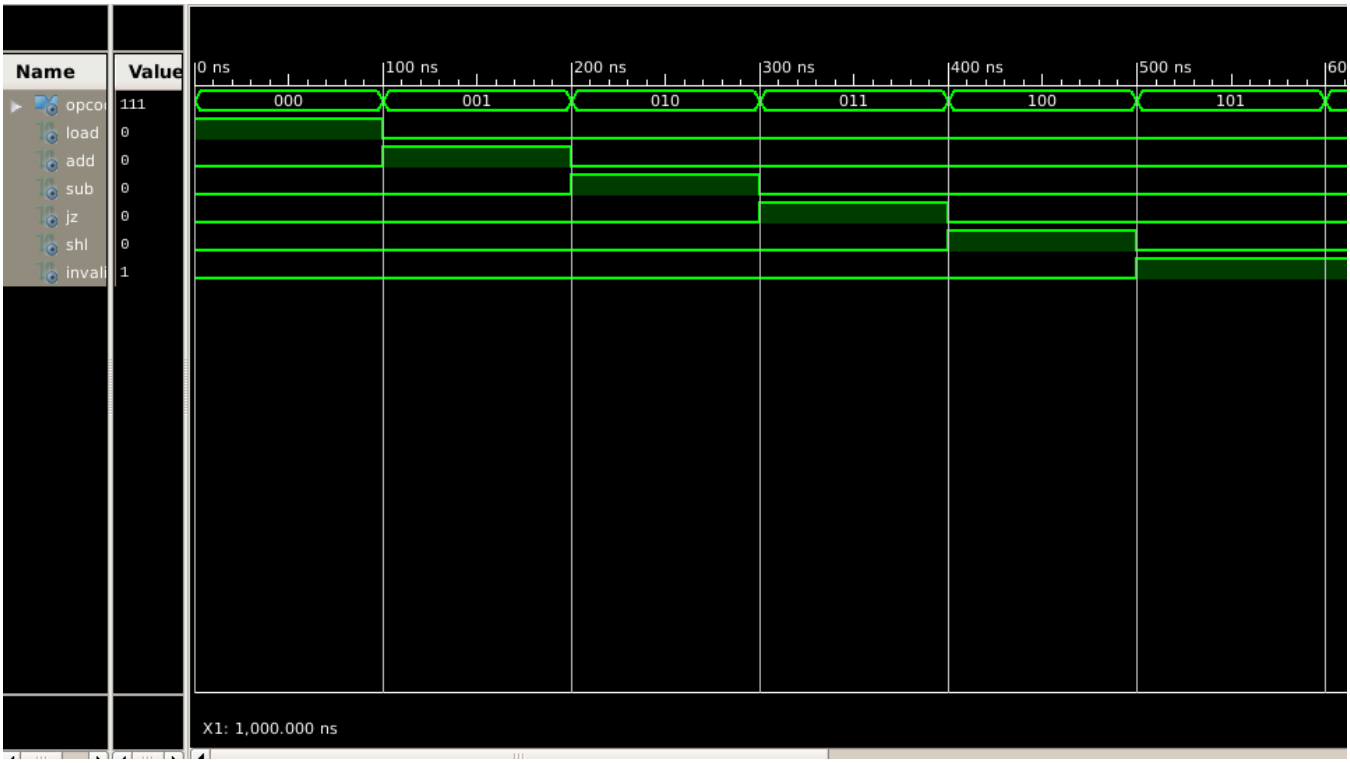


Figure 15: *Simulation waveform of control_unit*

LDI gives ACC_SRC = 0 and ACC_WE = 1. ADD, SUB and SHL give ACC_SRC = 1, ACC_WE = 1 and ALU_SEL = 00, 01 and 10. JZ 0x07 gives ACC_WE = 0, with PC_LOAD = 0 while Z = 0 and PC_LOAD = 1 once Z = 1. The NOP (0x000) gives ACC_WE = 0 and PC_LOAD = 0 even with Z = 1.

4.8 Top-level CPU (cpu_top)

cpu_top connects all blocks as shown in Figure 1: program counter (with the JZ load path), program ROM, control unit, ALU, ACC source multiplexer, accumulator, zero detector and the Z flip-flop. The stored Z is fed back to the control unit as the JZ condition. The outputs are Q = ACC, Z and PC; the current instruction is also brought out (INSTRUCTION_OUT) to make the waveform easier to read. All registers use the same CLK and the synchronous, active-high RESET.

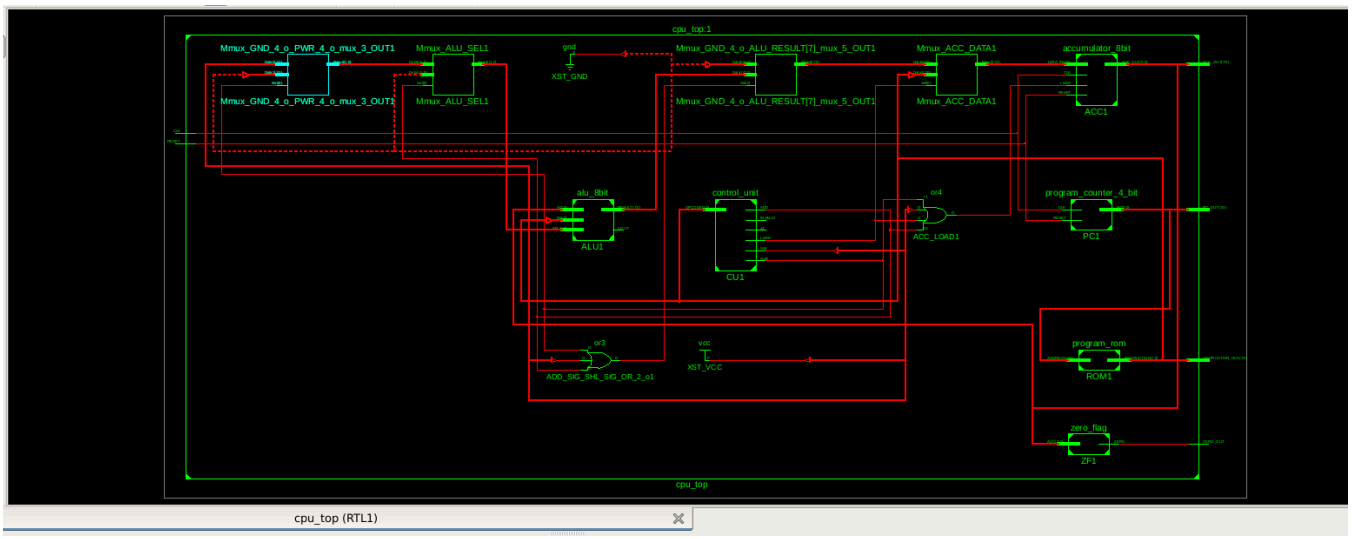


Figure 16: *Block symbol of cpu_top*

VHDL code: cpu_top.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Mini 8-bit accumulator CPU, Set 6 (LDI, ADD, SUB, JZ, SHL)
-- Single cycle: fetch, decode and execute in one clock period,
-- results are written at the rising edge of CLK.
entity cpu_top is
    Port (
        CLK             : in  STD_LOGIC;
        RESET           : in  STD_LOGIC;
        Q               : out STD_LOGIC_VECTOR(7 downto 0);   -- ACC
        Z               : out STD_LOGIC;                      -- zero flag
        PC              : out STD_LOGIC_VECTOR(3 downto 0);   -- program counter
        INSTRUCTION_OUT : out STD_LOGIC_VECTOR(11 downto 0)   -- current instruction
    );
end cpu_top;

architecture Structural of cpu_top is

    component program_counter_4_bit
        Port (
            CLK   : in  STD_LOGIC;
            RESET : in  STD_LOGIC;
            LOAD  : in  STD_LOGIC;
            D     : in  STD_LOGIC_VECTOR(3 downto 0);
            PC    : out STD_LOGIC_VECTOR(3 downto 0)
        );
    end component;

    component program_rom
        Port (
            ADDRESS     : in  STD_LOGIC_VECTOR(3 downto 0);
            INSTRUCTION : out STD_LOGIC_VECTOR(11 downto 0)
        );
    end component;

    component control_unit
        Port (
            INSTRUCTION : in  STD_LOGIC_VECTOR(11 downto 0);
            Z           : in  STD_LOGIC;
            OPERAND     : out STD_LOGIC_VECTOR(7 downto 0);
            ALU_SEL     : out STD_LOGIC_VECTOR(1 downto 0);
            ACC_SRC     : out STD_LOGIC;
            ACC_WE      : out STD_LOGIC;
            PC_LOAD     : out STD_LOGIC;
            LOAD_EN     : out STD_LOGIC;
            ADD_EN      : out STD_LOGIC;
            SUB_EN      : out STD_LOGIC;
            JZ_EN       : out STD_LOGIC;
            SHL_EN      : out STD_LOGIC
        );
    end component;

    component alu_8bit
        Port (
            A      : in  STD_LOGIC_VECTOR(7 downto 0);
            B      : in  STD_LOGIC_VECTOR(7 downto 0);
            SEL    : in  STD_LOGIC_VECTOR(1 downto 0);
            RESULT : out STD_LOGIC_VECTOR(7 downto 0);
            COUT   : out STD_LOGIC
        );
    end component;

    component accumulator_8bit
        Port (
            CLK     : in  STD_LOGIC;
            RESET   : in  STD_LOGIC;
            DATA_IN : in  STD_LOGIC_VECTOR(7 downto 0);
            LOAD    : in  STD_LOGIC;
            ACC_OUT : out STD_LOGIC_VECTOR(7 downto 0)
        );
    end component;

    component zero_flag
        Port (
            ACC  : in  STD_LOGIC_VECTOR(7 downto 0);
            ZERO : out STD_LOGIC
        );
    end component;

    component D_flip_flop
        Port (
            D     : in  STD_LOGIC;
            CLK   : in  STD_LOGIC;
            RESET : in  STD_LOGIC;
            Q     : out STD_LOGIC
        );
```

```
    end component;

    signal PC_VALUE    : STD_LOGIC_VECTOR(3 downto 0);
    signal INSTRUCTION : STD_LOGIC_VECTOR(11 downto 0);
    signal OPERAND     : STD_LOGIC_VECTOR(7 downto 0);
    signal ALU_SEL     : STD_LOGIC_VECTOR(1 downto 0);
    signal ACC_SRC     : STD_LOGIC;
    signal ACC_WE      : STD_LOGIC;
    signal PC_LOAD     : STD_LOGIC;
    signal JZ_TARGET   : STD_LOGIC_VECTOR(3 downto 0);
    signal Z_VALUE     : STD_LOGIC;
    signal LOAD_SIG    : STD_LOGIC;
    signal ADD_SIG     : STD_LOGIC;
    signal SUB_SIG     : STD_LOGIC;
    signal JZ_SIG      : STD_LOGIC;
    signal SHL_SIG     : STD_LOGIC;
    signal ACC_VALUE   : STD_LOGIC_VECTOR(7 downto 0);
    signal ALU_RESULT  : STD_LOGIC_VECTOR(7 downto 0);
    signal ALU_COUT    : STD_LOGIC;
    signal ACC_DATA    : STD_LOGIC_VECTOR(7 downto 0);
    signal ZERO_NEXT   : STD_LOGIC;
    signal NZ_D        : STD_LOGIC;   -- NOT Z, next value
    signal NZ_Q        : STD_LOGIC;   -- NOT Z, stored (resets to 0, so Z = 1)

begin

    -- 1. Program counter (PC <- PC + 1, or JZ target when PC_LOAD = 1)
    JZ_TARGET <= OPERAND(3 downto 0);

    PC1: program_counter_4_bit
        port map (
            CLK   => CLK,
            RESET => RESET,
            LOAD  => PC_LOAD,
            D     => JZ_TARGET,
            PC    => PC_VALUE
        );

    -- 2. Program ROM (combinational read)
    ROM1: program_rom
        port map (
            ADDRESS     => PC_VALUE,
            INSTRUCTION => INSTRUCTION
        );

    -- 3. Control unit (decoder + datapath control)
    CU1: control_unit
        port map (
            INSTRUCTION => INSTRUCTION,
            Z           => Z_VALUE,
            OPERAND     => OPERAND,
            ALU_SEL     => ALU_SEL,
            ACC_SRC     => ACC_SRC,
            ACC_WE      => ACC_WE,
            PC_LOAD     => PC_LOAD,
            LOAD_EN     => LOAD_SIG,
            ADD_EN      => ADD_SIG,
            SUB_EN      => SUB_SIG,
            JZ_EN       => JZ_SIG,
            SHL_EN      => SHL_SIG
        );

    -- 4. ALU: A = ACC, B = operand
    ALU1: alu_8bit
        port map (
            A      => ACC_VALUE,
            B      => OPERAND,
            SEL    => ALU_SEL,
            RESULT => ALU_RESULT,
            COUT   => ALU_COUT
        );

    -- 5. ACC data source: operand for LDI, ALU result otherwise
    ACC_DATA <= ALU_RESULT when ACC_SRC = '1' else OPERAND;

    -- 6. Accumulator (hold / load register)
    ACC1: accumulator_8bit
        port map (
            CLK     => CLK,
            RESET   => RESET,
            DATA_IN => ACC_DATA,
            LOAD    => ACC_WE,
            ACC_OUT => ACC_VALUE
        );

    -- 7. Zero flag: detector on the value being written,
    --    stored in a flip-flop together with ACC
    --    (JZ and NOP do not write, so Z keeps its value).
    ZF1: zero_flag
```

```
        port map (
            ACC  => ACC_DATA,
            ZERO => ZERO_NEXT
        );

    NZ_D <= (not ZERO_NEXT) when ACC_WE = '1' else NZ_Q;

    ZREG: D_flip_flop
        port map (
            D     => NZ_D,
            CLK   => CLK,
            RESET => RESET,
            Q     => NZ_Q
        );

    Z_VALUE <= not NZ_Q;

    -- 8. Outputs
    Q               <= ACC_VALUE;
    Z               <= Z_VALUE;
    PC              <= PC_VALUE;
    INSTRUCTION_OUT <= INSTRUCTION;

end Structural;
```

5. Top-Level Testbench and Results

The testbench generates a 20 ns clock, applies RESET for one rising edge and then runs one clock cycle for each executed instruction of the Set 6 program. One nanosecond after every edge it checks Q (ACC), Z and PC with assert statements against the expected Appendix B values, including the not-taken and the taken JZ. One extra cycle checks that the NOP at address 10 leaves ACC and Z unchanged. The complete testbench code follows.

Testbench code: cpu_top_tb.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY cpu_top_tb IS
END cpu_top_tb;

ARCHITECTURE behavior OF cpu_top_tb IS

    COMPONENT cpu_top
    PORT(
        CLK             : IN  std_logic;
        RESET           : IN  std_logic;
        Q               : OUT std_logic_vector(7 downto 0);
        Z               : OUT std_logic;
        PC              : OUT std_logic_vector(3 downto 0);
        INSTRUCTION_OUT : OUT std_logic_vector(11 downto 0)
    );
    END COMPONENT;

    -- Inputs
    SIGNAL CLK   : std_logic := '0';
    SIGNAL RESET : std_logic := '0';

    -- Outputs
    SIGNAL Q               : std_logic_vector(7 downto 0);
    SIGNAL Z               : std_logic;
    SIGNAL PC              : std_logic_vector(3 downto 0);
    SIGNAL INSTRUCTION_OUT : std_logic_vector(11 downto 0);

    CONSTANT CLK_period : time := 20 ns;

BEGIN

    uut: cpu_top PORT MAP (
        CLK             => CLK,
        RESET           => RESET,
        Q               => Q,
        Z               => Z,
        PC              => PC,
        INSTRUCTION_OUT => INSTRUCTION_OUT
    );

    -- Clock generation: 20 ns period
    CLK_process: PROCESS
    BEGIN
        CLK <= '0';
        WAIT FOR CLK_period / 2;
        CLK <= '1';
        WAIT FOR CLK_period / 2;
```

```
    END PROCESS;

    -- Set 6 test sequence (Appendix B)
    stim_proc: PROCESS
    BEGIN
        -- Cycle 0: RESET for one rising edge
        RESET <= '1';
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
        ASSERT Q = x"00" AND Z = '1' AND PC = "0000"
            REPORT "FAIL: reset state" SEVERITY ERROR;
        RESET <= '0';

        -- Cycle 1: PC 0, LDI 0x02
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
        ASSERT Q = x"02" AND Z = '0' AND PC = "0001"
            REPORT "FAIL: LDI 0x02" SEVERITY ERROR;

        -- Cycle 2: PC 1, SUB 0x01
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
        ASSERT Q = x"01" AND Z = '0' AND PC = "0010"
            REPORT "FAIL: SUB 0x01 at address 1" SEVERITY ERROR;

        -- Cycle 3: PC 2, JZ 0x07 (not taken, Z = 0)
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
        ASSERT Q = x"01" AND Z = '0' AND PC = "0011"
            REPORT "FAIL: JZ 0x07 not taken" SEVERITY ERROR;

        -- Cycle 4: PC 3, SUB 0x01 -> zero result
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
        ASSERT Q = x"00" AND Z = '1' AND PC = "0100"
            REPORT "FAIL: SUB 0x01 at address 3" SEVERITY ERROR;

        -- Cycle 5: PC 4, JZ 0x07 (taken, Z = 1)
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
        ASSERT Q = x"00" AND Z = '1' AND PC = "0111"
            REPORT "FAIL: JZ 0x07 taken" SEVERITY ERROR;

        -- Cycle 6: PC 7, LDI 0x21 (addresses 5 and 6 were skipped)
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
        ASSERT Q = x"21" AND Z = '0' AND PC = "1000"
            REPORT "FAIL: LDI 0x21" SEVERITY ERROR;

        -- Cycle 7: PC 8, SHL
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
        ASSERT Q = x"42" AND Z = '0' AND PC = "1001"
            REPORT "FAIL: SHL" SEVERITY ERROR;

        -- Cycle 8: PC 9, ADD 0x01
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
        ASSERT Q = x"43" AND Z = '0' AND PC = "1010"
            REPORT "FAIL: ADD 0x01" SEVERITY ERROR;

        -- Cycle 9: PC 10 holds 0x000 (NOP): ACC and Z unchanged
        WAIT UNTIL rising_edge(CLK); WAIT FOR 1 ns;
        ASSERT Q = x"43" AND Z = '0' AND PC = "1011"
            REPORT "FAIL: NOP" SEVERITY ERROR;

        REPORT "Set 6 CPU testbench completed." SEVERITY NOTE;
        WAIT;
    END PROCESS;

END behavior;
```

5.1 Simulation waveform

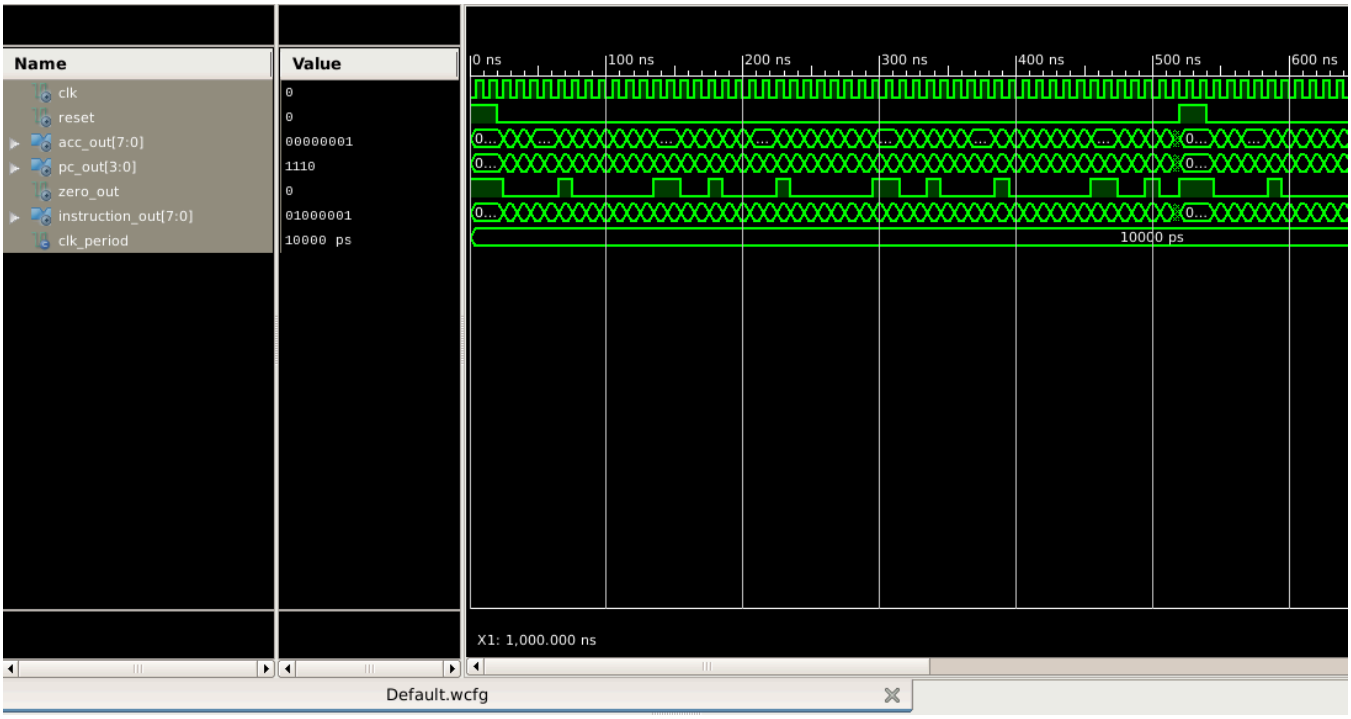


Figure 17: *Waveform of cpu_top running the Set 6 program (CLK, RESET, PC, current instruction, Q = ACC, Z)*

RESET is high at the first rising edge (10 ns), so PC = 0, ACC = 0x00 and Z = 1. The program then runs one instruction per rising edge:

- **LDI 0x02** loads ACC = 0x02 and Z becomes 0.
- **SUB 0x01** gives 0x02 - 0x01 = 0x01, Z = 0.
- **JZ 0x07** is not taken because Z = 0: PC simply goes from 2 to 3, ACC stays 0x01.
- **SUB 0x01** gives 0x01 - 0x01 = 0x00, so Z becomes 1.
- **JZ 0x07** is taken because Z = 1: PC jumps from 4 to 7 and ACC is unchanged. The two ADD 0xAA instructions at addresses 5 and 6 are skipped.
- **LDI 0x21** loads ACC = 0x21 (0010 0001) and Z = 0.
- **SHL** shifts ACC left: 0x21 -> 0x42 (0100 0010).
- **ADD 0x01** gives ACC = 0x43 and PC = 10.
- Address 10 holds 0x000 (NOP): ACC stays 0x43, Z stays 0, PC becomes 11.

5.2 Result check against Appendix B

Table 4: *Simulated values after each rising edge compared with the Appendix B table for Set 6*

Cycle	PC	Instruction executed	ACC (hex)	ACC (binary)	Z	Next PC	Status
0	-	RESET = 1	0x00	00000000	1	0	Pass
1	0	LDI 0x02	0x02	00000010	0	1	Pass
2	1	SUB 0x01	0x01	00000001	0	2	Pass
3	2	JZ 0x07 (not taken)	0x01	00000001	0	3	Pass
4	3	SUB 0x01	0x00	00000000	1	4	Pass
5	4	JZ 0x07 (taken)	0x00	00000000	1	7	Pass
6	7	LDI 0x21	0x21	00100001	0	8	Pass

Cycle	PC	Instruction executed	ACC (hex)	ACC (binary)	Z	Next PC	Status
7	8	SHL	0x42	01000010	0	9	Pass
8	9	ADD 0x01	0x43	01000011	0	10	Pass
9	10	NOP (0x000, extra check)	0x43	01000011	0	11	Pass

All eight program cycles match the expected ACC, Z and next-PC values of Appendix B (Set 6), including the not-taken JZ at address 2 and the taken JZ at address 4. The testbench finished with the message "Set 6 CPU testbench completed." and no assertion failure. The module testbenches (D flip-flop, register, ALU, zero flag, accumulator, program counter, ROM, decoder and control unit) also finished without any assertion failure.

6. Challenges Faced

- **Following the exact instruction format.** The first version of the CPU already had the Set 6 instructions (LDI, ADD, SUB, JZ, SHL), but used 8-bit instructions with a 3-bit opcode (000 - 100) and a 5-bit operand. It was changed to the required 12-bit format with the official 4-bit opcodes (0001, 0010, 0011, 1100, 1000) and an 8-bit operand, and the ROM was filled with the Appendix B program of Set 6.
- **ALU select did not match the top level.** In the first ALU the select codes meant ADD / NAND / NOT / zero, while cpu_top used them as ADD / SUB / SHL, so SUB actually produced NOT ACC and SHL always produced 0. The new ALU defines SEL = 00/01/10 as ADD/SUB/SHL, and the control unit generates exactly these codes.
- **Subtraction without a separate subtractor.** SUB reuses the 8-bit adder: B is XORed with SUB_MODE and SUB_MODE is used as the carry-in, giving A + NOT(B) + 1 = A - B (modulo 256).
- **Program counter had no load input.** The first PC could only count, so JZ was decoded but never jumped. A next-PC multiplexer (jump target or PC + 1) was added in front of the flip-flops, and the control unit drives it with PC_LOAD = JZ_EN AND Z.
- **Synchronous reset.** The original D flip-flop and program counter had an asynchronous reset, while the specification requires a synchronous one. The reset was moved inside the rising_edge(CLK) branch, and the testbenches check that RESET between clock edges does not change Q.
- **Z flag as a register.** Z was first a combinational function of ACC. It is now stored in a flip-flop that is written only together with ACC, so JZ and NOP keep it. Because the flip-flop resets to 0 but Z must be 1 after RESET, the flip-flop stores NOT Z.
- **VHDL-93 restrictions in ISE 14.7.** Expressions are not allowed as actuals in port maps, so combined signals such as PC_LOAD, SUB_MODE and the jump target operand[3:0] are declared as separate signals.
- **Testbench timing.** Outputs are sampled 1 ns after each rising edge so that the checks never race with the register updates. An empty file named bit-8_full_adder.vhd (a hyphen is not legal in a VHDL name) was removed from the project.

7. Conclusion

A single-cycle 8-bit accumulator CPU for Set 6 (LDI, ADD, **SUB**, **JZ**, **SHL**) was designed in VHDL using a modular, gate-level hierarchy built from NAND gates. The design contains separate entities for the program counter, program ROM, instruction decoder, control unit, ALU, accumulator and zero flag, connected in a top-level cpu_top entity with inputs CLK and RESET (synchronous, active-high) and outputs Q[7:0], Z and PC[3:0]. The Appendix B program was stored in the ROM and executed by the testbench: ACC, Z and PC

match the expected values on every cycle, including the not-taken and the taken JZ. The project showed how a complete processor can be built step by step from one primitive gate and verified at each level.

8. Submitted Files

- **VHDL entities:** NAND_gate, not_gate, and_gate, or_gate, nor_gate, xor_gate, xnor_gate, half_adder, full_adder, bit_8_full_adder, D_flip_flop, register_8_bit, alu_8bit, zero_flag, accumulator_8bit, program_counter_4_bit, program_rom, instruction_decoder, control_unit, cpu_top (.vhd).
- **Testbenches:** one _tb.vhd file for every entity, including cpu_top_tb.vhd for the Set 6 program.

9. GitHub Link

https://github.com/Rifah03/VLSI-ISE-14.7